Computer Science Extended Essay

Title: An investigation into the optimisation of hyperparameters of machine learning models using Genetic Algorithms

Research Question: To what extent can genetic algorithms optimise the hyperparameters of an Ensemble Gradient Boosting Regressor model to predict housing prices compared to Bayes Search and Random Search?

Word Count: 3, 990 words

Session: May 2025

Table of Contents

1. Introduction……. 1
2. Scope and significance of this research…….…… 2
3. Background Research……. 2
    a. Definitions of Key Terms used throughout the paper…….…. 2
    b. Hyperparameter Optimisation……. 4
    c. The working of Random Search……. 4
    d. The working of Bayes Search……. 5
    e. Definition of key terms used for the Genetic Algorithm…… 5
    f. The working of Genetic Algorithm……. 8
4. Hypothesis…….. 9
5. Methodology……... 9
    a. Research Workflow……. 9
    b. Datasets Used…… 10
    c. Description of Machine Learning Algorithm Used…….. 11
    d. Description of Hyperparameters optimised and the ranges used……. 12
    e. GA Implementation…….. 14
    f. Building the GA…… 14
        i. Hyperparameter encoding……. 17
        ii. Elitism Operator…… 17
        iii. Selection Operator……. 17
        iv. Crossover Operator…….. 20
        v. Mutation Operator……. 21
        vi. Dynamic Population Scaling……. 22
        vii. GA Parameter Evaluation……. 23
6. Analysis and Evaluation……... 25
    a. Dataset Characteristics……. 25
    b. Small vs Large Search Spaces……. 28
    c. Evaluations based on Time Constraints……... 29
        i. Relative Short Timeframes…….. 29

ii. Relative Moderate Timeframes…… 31
iii. Relative Extended Timeframes…… 32
d. Drawbacks of each algorithm……... 33
i. Genetic Algorithm…… 33
ii. Bayes Search……. 34
iii. Random Search……. 35
7. Limitations and Conclusion……..….35
a. Limitations of this Research for Further Investigation…….. 35
b. Conclusion…… 36
8. Works Cited……...37
9. Appendix 1…… 39
a. Primary data collected from different configurations of the GA…….. 39
b. Primary data collected from different configurations of BS……. 49
c. Primary data collected from different configurations of Random Search…… 55
10. Appendix 2…… 57
a. GA Implementation - Services Code…… 57
b. GA Implementation - Core Functionality……. 63

1 - Introduction

Genetic Algorithms (GAs) are specialised heuristic algorithms that prioritise finding good enough solutions over the best solution, often used for complex problems where evaluating each solution is too time-consuming or impractical. Drawing inspiration from genetics, the algorithm employs selection, crossover, and mutation mechanisms to create its foundational operations.

Hyperparameter optimisation has been a challenge in the machine learning industry since its inception, even though it plays a crucial role in the final accuracy. Having faced the same issue when optimising a K-Nearest Neighbor algorithm, manually finding the right set of hyperparameters can be time-consuming for data scientists, often requiring much experience working with the model. Therefore, the need for automated optimisation techniques is ever-increasing.

Genetic Algorithms (GAs), introduced by John Holland in the 1970s, employ a population-based approach that assumes a correlation between the quality of neighbouring solutions. By combining the "genetic material" of high-quality solutions, GAs generate offspring that can potentially outperform their parents. Often, these solutions come from different regions of the search space, therefore reducing the risk of premature convergence. GAs are also one of the most prevalent metaheuristic approaches for hyperparameter optimisation (Yang and Shami 4). Given the multi-modal nature of the hyperparameter search space, other popular heuristic algorithms, such as Hill Climbing and Simulated Annealing, may struggle to navigate the search space effectively.

2 - Scope and Significance of this research

This research aims to prove the feasibility of GAs in the hyperparameter optimisation industry. Due to computational and time constraints, this GA will only focus on the Ensemble Gradient Boosting Regressor model for predicting housing prices in the Melbourne and California datasets. It aims to demonstrate the best optimisation algorithm from GA, Random Search (RS), and Bayes Search (BS) under different conditions. While this research is limited to one model and two datasets, the methodology can be extended to various models, datasets and optimisation problems across multiple fields.

3 - Background Research

3.a - Definition of Key Terms used throughout the paper

Table 1: Definitions of Key Terms in Hyperparameter Optimization and Search Algorithms

Hyperparameters	Values that determine the learning process of a model, and how patterns are extracted from input data. The user sets the hyperparameters' values before the model's training even begins (Pandian). They can be thought of as knobs for the model that significantly impact its performance and accuracy.
Parameters (weight and biases)	They are learned or estimated from the training dataset as the algorithm learns the pattern between input features and the targets (Pandian).

Curse of Dimensionality	Refers to the challenges that arise as the number of parameters to optimise increases, as it exponentially increases the number of potential solutions to evaluate.
Sample	Represents a single data instance that the model uses for training or prediction.
Underfitting	A model too simple to capture patterns in data which leads to poor performance on both training and test sets.
Overfitting	A model that is excessively complex and fits noise in training data, resulting in poor generalization of unseen data in real-life scenarios.
Objective Function	A function that evaluates solutions in an optimization problem, guiding the search by measuring performance.
Optimisation Problem	A problem where the goal is to find the best solution by maximizing or minimizing an objective function.
Minimisation Optimisation Problem	The goal of the minimisation optimisation problem is to reduce the value returned by the objective function.
Maximisation Optimisation Problem	The goal of the maximisation optimisation problem is to increase the value returned by the objective function.

3.b - Hyperparameter Optimisation

Hyperparameter optimisation is the process of finding the correct set of hyperparameters in a reasonable amount of time to achieve maximum performance (David). It is often considered the trickiest part of building machine learning models. The traditionally used methods are Trial and Error, Grid Search, RS, and BS.

Trial and error relies entirely on the knowledge of the data scientist and requires a lot of experience. It can still be time-consuming to look through many potential configurations manually (Koehrsen). GridSearch exhaustively searches through all possible values in a defined grid, relying on data scientists' knowledge to define a grid that can offer an optimal solution (Koehrsen).

Sections 3.c and 3.d explore the two algorithms compared in this study against GA, RS and BS, in more detail.

3.c - The working of Random Search

RS is considerably more efficient than Grid Search despite both being exhaustive. This is partly because RS takes random values for each hyperparameter, effectively exploring the search space (Koehrsen). It is recommended for search spaces with fewer hyperparameters as it provides a simple and effective solution for optimisation. However, in high-dimensional search spaces, RS misses effective solutions in the search space, as there are too many solutions to consider.

3.d - The working of Bayes Search

BS uses the Bayes Theorem to find an optimal solution. It employs a surrogate model, usually the Gaussian Process, to predict a hyperparameter configuration's performance before testing it (Frazier 1). The surrogate model is first built by testing random sets of hyperparameters. Based on the surrogate model, an acquisition function is created, which determines the next set of values for the hyperparameters to evaluate in an iterative process (Frazier 1).

However, BS is a computationally expensive method and is highly influenced by the acquisition function and initial set of hyperparameters that are evaluated at the beginning of the process. The surrogate model often assumes a smooth and continuous curve across the search space. However, skewness can distort this smoothness and outliers amplify this distortion. This can result in a suboptimal performance by the algorithm.

3.e - Definitions of key terms used for the Genetic Algorithm

Table 2: Important keywords related to the GA

Chromosome	An encoded string of 0s and 1s representing solutions in the GA. The process of encoding for this GA is examined later. The encoded hyperparameter configuration of the model serves as the chromosome.
Genotype	The encoded configuration of bits (0s and 1s) in a chromosome (Mitchell 5).

Phenotype	Refers to an individual solution's physical and observable characteristics (Mitchell 5). The decoded hyperparameter configuration from a chromosome is the phenotype of the chromosome for this GA.
Fitness	The ability of an organism to survive in its environment determines the probability that the organism will live to reproduce (Mitchell 5).
Fitness Value	The numerical value assigned to a solution that determines its fitness (Mitchell 5). In this GA, the scoring metric passed, such as mean absolute error, will be the fitness function. In a minimisation optimisation problem, such as the one explored in this study, the lower the fitness value of a solution, the higher the chances of the solution's chances of reproduction.
Search Space	An n-dimensional space containing the whole collection of candidate solutions (Mitchell 6). Here, n represents the number of hyperparameters being optimised. Each dimension holds the possible values for a single hyperparameter. For instance, a hyperparameter with three possible values exists in a discrete space, meaning its dimension can only take those three values, unlike continuous parameters (such as hyperparameters that accept decimal values) where the dimension can take an infinite set of numbers.

Fitness Landscape	An (n + 1)-dimensional space that maps a solution with its fitness value. This can form hills, peaks, and valleys and features similar to physical landscapes (Mitchell 6).
Population	A set of potential solutions to the problem.
Genetic Operators	Mechanisms in Genetic Algorithms, including selection, crossover, and mutation, that modify and evolve populations to achieve improved quality solutions.
Generation	One iteration of the full GA process which processes a single population. The genetic operators are applied to the individuals of a generation to create individuals for the next generation.
Convergence	The point at which evolution slows or stops because nearly all individuals in the population occupy the same region of the search space, solely leading to exploitation of that area.
Premature Convergence	Occurs when the population gets stuck on a suboptimal solution due to a variety of factors, including higher selection pressure, low mutation rate, etc.
Fitness scaling	Normalization of fitness values to prevent the dominance of the fittest individuals. Normalization can help prevent premature convergence.

Selection pressure	The degree to which fitter individuals are preferred over less fit individuals during the selection process and can determine the balance between exploration and exploitation.

3.f - The working of Genetic Algorithm

Integrating logic inspired by nature into a random process in the GA boosts the success rate of finding an optimal solution by a significant margin compared to traditionally used algorithms. As it does not work on any predefined assumptions, it performs well on unconventional datasets. Additionally, this GA's population-based approach decreases the likelihood of being stuck in a local optimum, perfect for the multimodal nature of any hyperparameter search space. Parallelism can be used to evaluate different solutions simultaneously, although it is not implemented in this GA.

However, GAs require careful tuning of parameters such as selection rates, mutation rates, population size, generation count, elitism rate, etc. These parameters significantly influence the algorithm's performance and convergence rate. Moreover, GAs are computationally expensive due to the large number of iterations and evaluations of different hyperparameter configurations required, particularly for high-dimensional search spaces.

4 - Hypothesis

Due to the maintenance of a large population for multiple generations, the GA will be more computationally expensive than BS and RS. However, this is offset by the GA’s ability to escape

a local optima easily, unlike BS which can be misguided especially in a skewed or complex dataset, leading to significant improvements.

Additionally, the modelled GA offers more flexibility in choosing key values of the algorithm such as population size, mutation rate, etc. While this requires more expertise from the user, the average expected performance improvement is expected to be significant over Bayes and RS for this reason, especially in larger search spaces.

5 - Methodology

5.a - Research Workflow

The GA was implemented from scratch, with the code available in Appendix 2. It comprises two sections: helper functions essential for the GA's operation, which can be found in Appendix 2 under Section 10.a, and the core GA functionality which can be found under Section 10.b. An in-depth explanation of the functionality of the GA can be found under sections 5.e and 5.f.

It was followed by the selection of the datasets and their models. The hyperparameters and their ranges were then selected to best represent real-world scenarios. A detailed explanation is given in sections 5.b to 5.d.

Each configuration of GA and BS was run thrice, and the best run has been included in Appendix 1, under sections 9.a and 9.b respectively. RS was only run once since it does not rely on logic, and the best solution’s loss function value is included in Appendix 1, under section 9.c.

For each generation of the GA, key metrics such as the highest, lowest, and average fitness values were recorded, along with all information about the candidate solutions that were used by the GA.

For BS, the inbuilt method *bayes_optimizer.cv_results_.items()* was used to store details for each iteration of the run, which includes all the values for key variables. Since RS does not rely on logic, only the details of the best solution found were stored.

Line graphs were drawn for GA and BS to show individual fitness progression throughout the generations/iterations, and a bar graph for RS to show the loss value of the best solution found in the run.

5.b - Datasets Used

California and Melbourne Housing Datasets were used to test the optimisation algorithms. California dataset was taken from the default datasets provided by Scikit-learn. The Melbourne dataset was taken from the dataset posted by user anthonypino (Pino). The difference in various economic and social conditions in the datasets ensures a fair representation of real-world scenarios. The datasets used have also been widely tested, providing a consistent and credible basis for evaluation. The training and testing dataset is split into 70:30, suitable for the selected datasets (Theobald 46).

5.c - Description of Machine Learning Algorithm used

Gradient Boosting Regressor is an ensemble model of decision trees, i.e., it combines the output of multiple decision trees to produce a more accurate output ("Gradient Boosting in ML"). This model was chosen for optimisation due to its wide use in predicting housing prices. It also uses numerous hyperparameters, giving a large scope for improvement and optimisation.

Table 3: Key Terms for Gradient Boosting Regressor Algorithm

Decision Tree	Weak learners that make predictions by splitting data into branches based on features.
Layers	Represent different stages where each new decision tree learns from the mistakes of the previous ones, gradually improving accuracy.
Splits	A decision point in a tree where the data is divided based on a specific feature value, helping the model make better predictions.
Internal Node	A decision-making node in the tree that is not a leaf node. It splits the data based on a condition.
Leaf Node	The last node of a decision tree which provides the model's prediction or output.

5.d - Description of Hyperparameters optimised and the ranges used

The following hyperparameters of the Gradient Boosting Regressor model were optimised in all three algorithms.

Table 4: Description of Hyperparameters optimised and the ranges used

Hyperparameter	Description (“GradientBoostingRegressor”)	Range (Small search space) (all-inclusive)	Range (Large Search Space) (all-inclusive)
n_estimators	Number of decision trees in the model. A higher value generally improves performance but increases computational time and might lead to overfitting, while a smaller value might not capture the patterns in the dataset.	150 - 300	100 - 600
learning_rate	The step size for updating the contributions of individual decision trees in the ensemble. Smaller values enable gradual learning, requiring more trees and time to converge, while larger values lead to faster but unstable convergence.	0.1 - 0.3	0.01 - 0.3
max_depth	Maximum number of layers (depth) of each tree. Deeper trees capture more complexity but might overfit the data.	5 - 30	5 - 50
min_samples_split	Minimum number of samples required to split an internal node. Higher values prevent splitting on small sample sizes, reducing overfitting.	4 - 10	2 - 20

min_samples_leaf	Minimum number of samples required in a leaf node. Higher values make the model less sensitive to small data variations, preventing overfitting.	4 - 10	2 - 20
max_features	Proportion of features considered when looking for the best split. Smaller values reduce overfitting but may limit model capacity.	0.1-0.9	0.05 - 1.0
Loss Function	Measures how well a model's prediction matches the actual values. The goal of the ensemble model is to minimise the loss function with the given set of hyperparameters and the trees. Optimization algorithms, such as the GA, aim to identify the optimal set of hyperparameters that minimize the loss function for the ensemble model.	huber, squared_error, quantile	huber, squared_error, quantile

Two hyperparameter ranges were used to test how the algorithms perform under different conditions. The first range had 1.55×10^7 unique solutions, while the second one had almost 7.18×10^{10} unique solutions, nearly 5000 times the size of the first, by extending the boundaries of permissible values for the hyperparameters. With the equipment used, for evaluating all the solutions in the Melbourne dataset, it would take approximately 3 years for the small range and

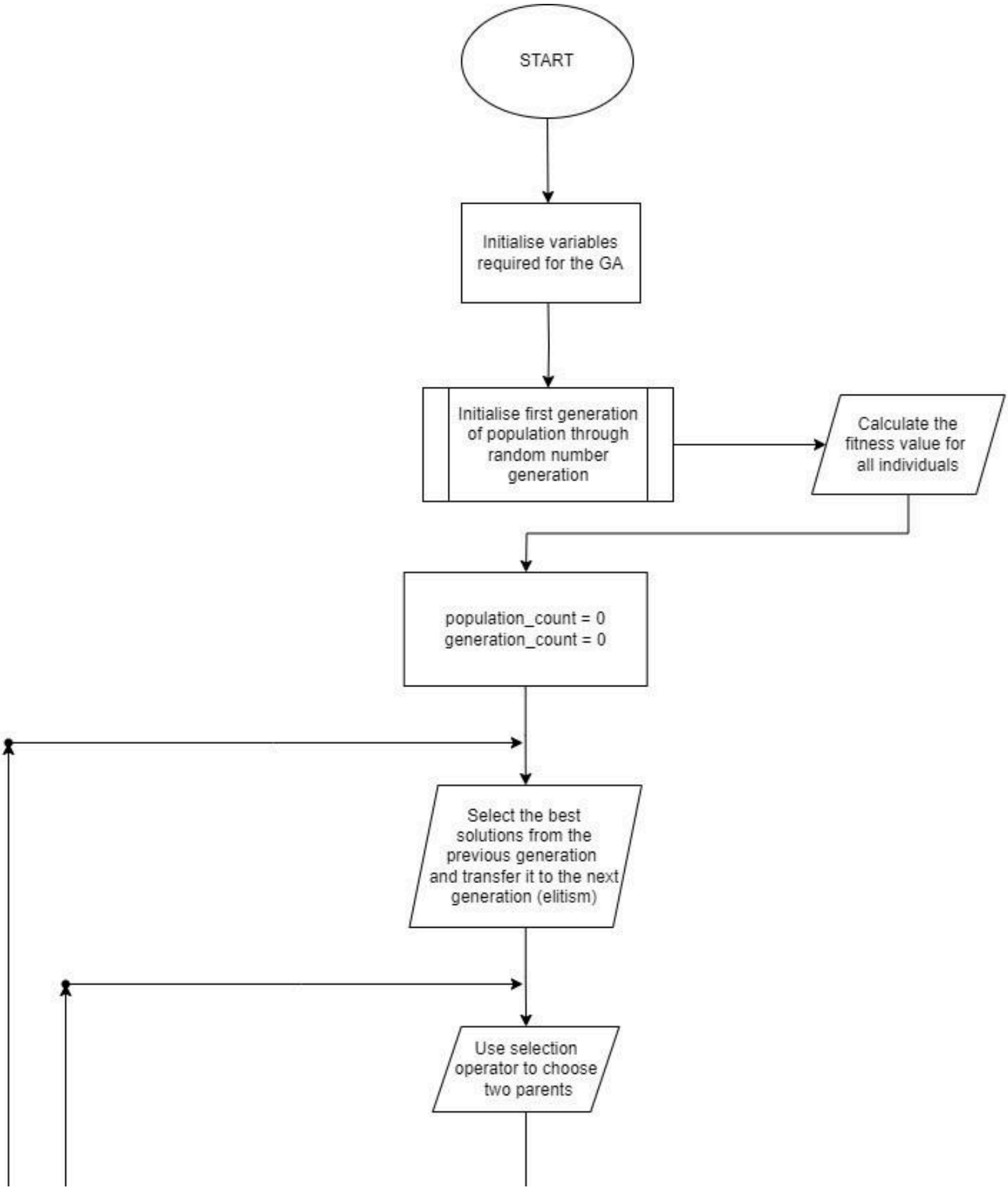
31000 years for the large range. For evaluating all solutions in the California dataset, it would take almost 4.5 years for the small range and 45500 years for the large range. Evaluating all solutions for any scenario is quite impossible.

5.e - GA Implementation

The GA was implemented in Python due to its extensive functionalities and library support. The complex data structures and intricate algorithms— such as reverse indexing and linked lists— required could be easily modelled in native Python, thus reducing the number of dependencies and improving the efficiency of the overall process.

5.f - Building the GA

The flowchart in Figure 1 below illustrates the GA process step-by-step, providing a visual overview of each stage. The operations are explained in more detail in the following sections.


START
Initialise variables required for the GA
Initialise first generation of population through random number generation
Calculate the fitness value for all individuals
population_count = 0
generation_count = 0
Select the best solutions from the previous generation and transfer it to the next generation (elitism)
Use selection operator to choose two parents

Continued

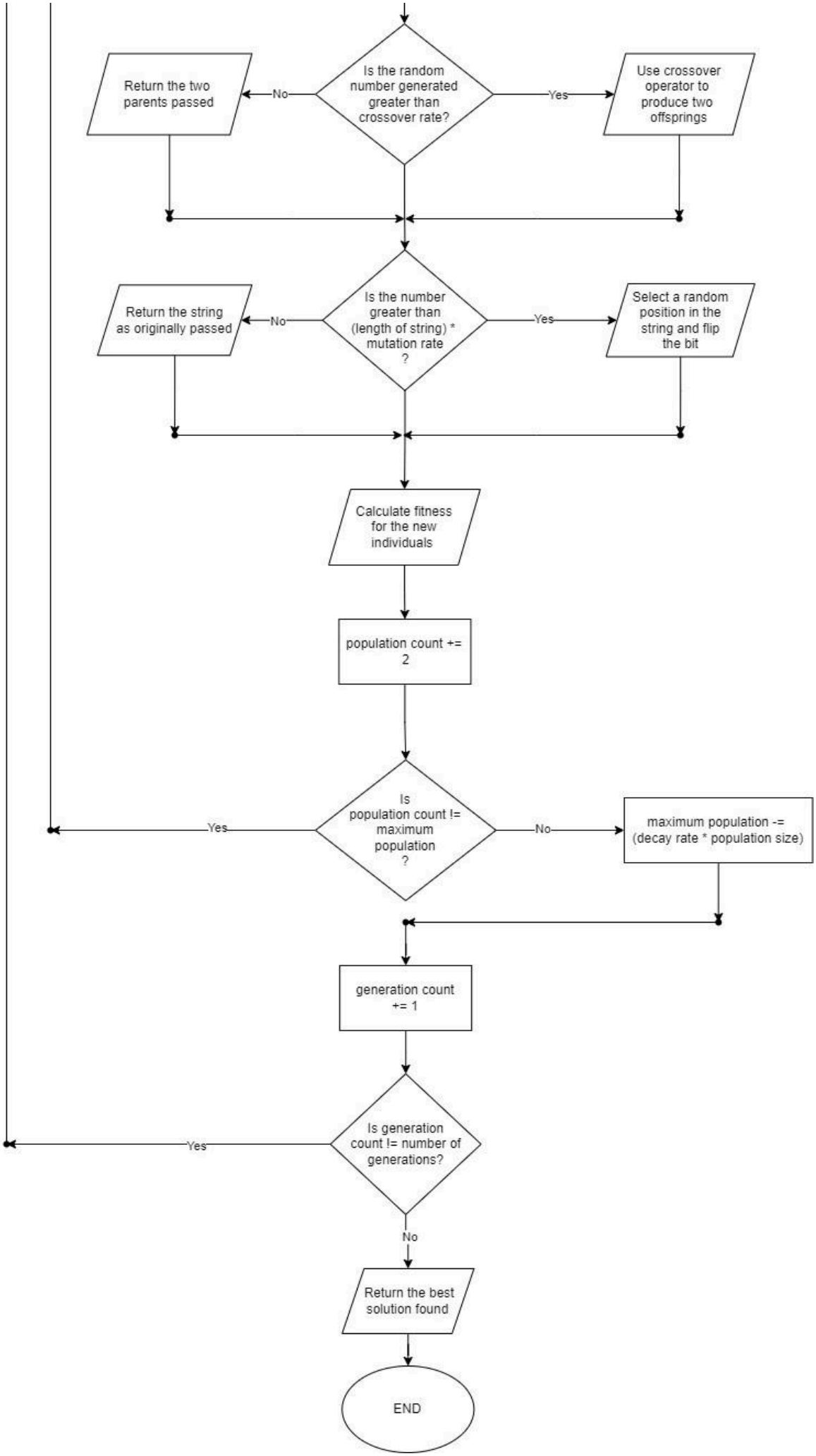

Is the random number generated greater than crossover rate?
No
Return the two parents passed
Yes
Use crossover operator to produce two offsprings
Is the number greater than (length of string) * mutation rate ?
No
Return the string as originally passed
Yes
Select a random position in the string and flip the bit
Calculate fitness for the new individuals
population count += 2
Is population count != maximum population ?
Yes
No
maximum population -= (decay rate * population size)
generation count += 1
Is generation count != number of generations?
Yes
No
Return the best solution found
END

Figure 1: Flowchart of the Genetic Algorithm Optimization Process (Created with draw.io)

5.f.i - Hyperparameter Encoding

The hyperparameters are encoded as a single string of 0s and 1s to make the functioning of genetic operators as fundamental as possible. Values of different data types are encoded through different procedures to make the chromosome succinct without losing valuable meaning, following the principle of minimal alphabets (Goldberg 80). All values of the same hyperparameter use the same number of bits in the string to ensure values do not lose their integrity after decoding, as per the principle of meaningful building blocks (Goldberg 80). Following these two principles ensures the search for the optimal solution is robust and efficient.

5.f.ii - Elitism operator

The elitism operator ensures that the best individuals in a population are passed on to the next generation (Mitchell 126). Thus, the population's quality does not degrade over generations as the best solutions from a generation are not lost. This operator also guides convergence, as the same individuals are selected and crossed over in the next generations until better solutions replace them.

5.f.iii - Selection Operator

As the first step in the GA process, the selection operator plays a pivotal role in the entire process by choosing the parents for subsequent crossover, thus making factors like fitness scaling and selection pressure crucial.

As such, Rank-based selection (Grefenstette 1) allows for the manual adjustment of selection pressure according to the number of generations and the search space size. It eliminates the need for fitness scaling by assigning ranks to the individuals, where the fitness function for ranking is obtained using the following formula (Grefenstette 1):

$$\phi_{(a_i)} = \delta f_{(a_i)}$$

$$\delta = +1 \; for \; maximization \; and \; -1 \; for \; minimization \; problems$$

$$\phi_{(a_i)} = Fitness \; Function \; upon \; which \; the \; ranks \; are \; evaluated$$

$$f_{(a_i)} = Objective \; function, \; such \; as \; mean \; absolute \; error \; from \; Scikit-learn$$

The best solution will have the rank of (size of population - 1) and the worst solution will have the rank of 0. In a minimisation problem, the best solution is the one with the lowest fitness, whereas for a maximisation problem, the best solution is the one with the highest fitness. The formula below (Grefenstette 2) can be used to obtain the probability of being selected for an individual based on rank:

$$Probability_{rank}(i) = \frac{\alpha_{rank} + [rank(i)/(\mu - 1)](\beta_{rank} - \alpha_{rank})}{\mu}$$

$$\alpha_{rank} = Number \; of \; offspring \; allocated \; to \; the \; worst \; individual$$

$$\beta_{rank} = Number \; of \; offspring \; allocated \; to \; the \; best \; individual$$

$$\mu = Size \; of \; the \; population$$

$$1 \leq \beta_{rank} \leq 2 \;\; , \;\; \alpha_{rank} = 2 - \beta_{rank}$$

$$rank(i) = Rank \; of \; the \; individual \; being \; evaluated$$

β_{rank} and α_{rank} directly influences the selection pressure. Higher selection pressure, achieved with a higher β_{rank}, benefits cases when the number of generations is low or the search space is too large for the GA to converge. Lower selection pressure will balance exploration and exploitation and benefits from a higher generation count or smaller search space to explore major areas of the search space for a higher probability of finding an optimal solution.

A roulette wheel selection is then used to select the parents based on the intermediate sum of the probabilities. In roulette wheel selection, each individual is given a slice of the roulette wheel (Mitchell 10). The area of the slice assigned to a individual is proportional to the probability found through the formula above for that individual.

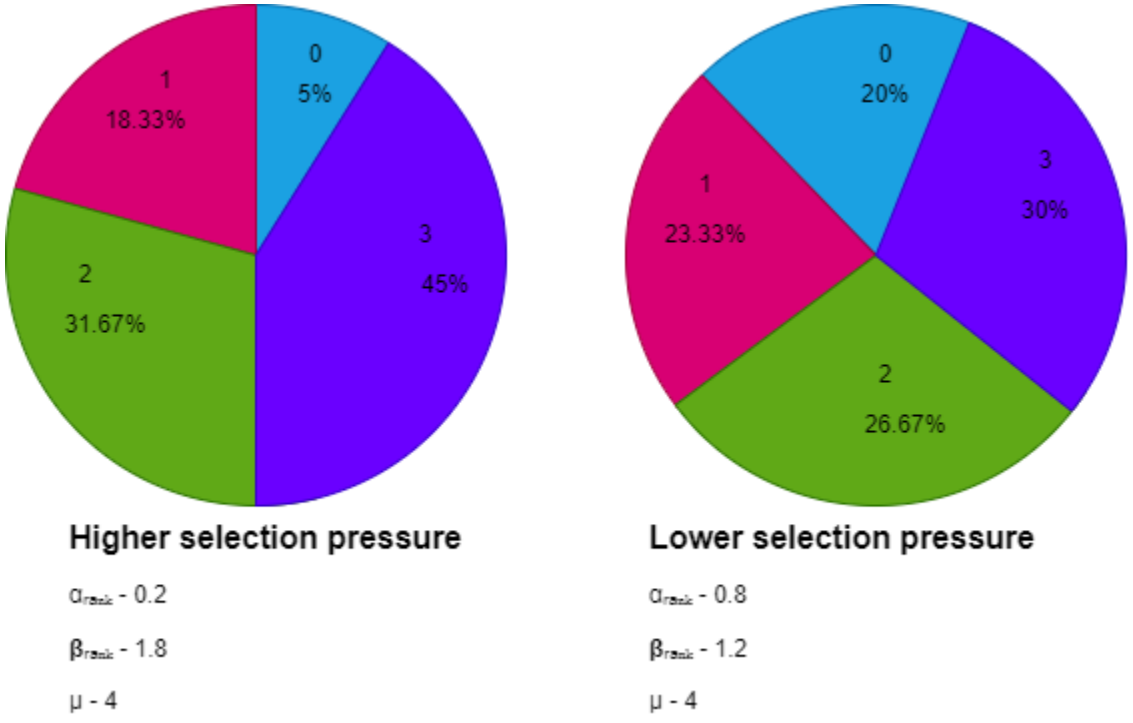


Figure 2: Roulette Wheel Selection, the best solution gets the most number of offspring in the next generation (Created with draw.io)

As observed from Figure 2, higher selection pressure favours the fitter individuals, accelerating convergence. Conversely, lower selection pressure gives all individuals an even chance of being selected.

5.f.iv - Crossover operator

The crossover operator creates two offspring by crossing over the genetic material of the two parents selected. A hybrid of multi-point and single-point crossover is used in this GA. The entire chromosome is divided into blocks at pre-designated crossover points, where each block represents the value for one hyperparameter. These blocks are then crossed over using single-point crossover to create two new offspring.

This algorithm leverages the fact that each hyperparameter is a core building block of the model and significantly influences its accuracy. Thus, a crossover between good enough hyperparameters for each configuration is expected to yield better results. Additionally, the traditionally used single-point crossover does not produce enough diversity or loses too much valuable information. The hybrid crossover is shown in Figure 3 below:

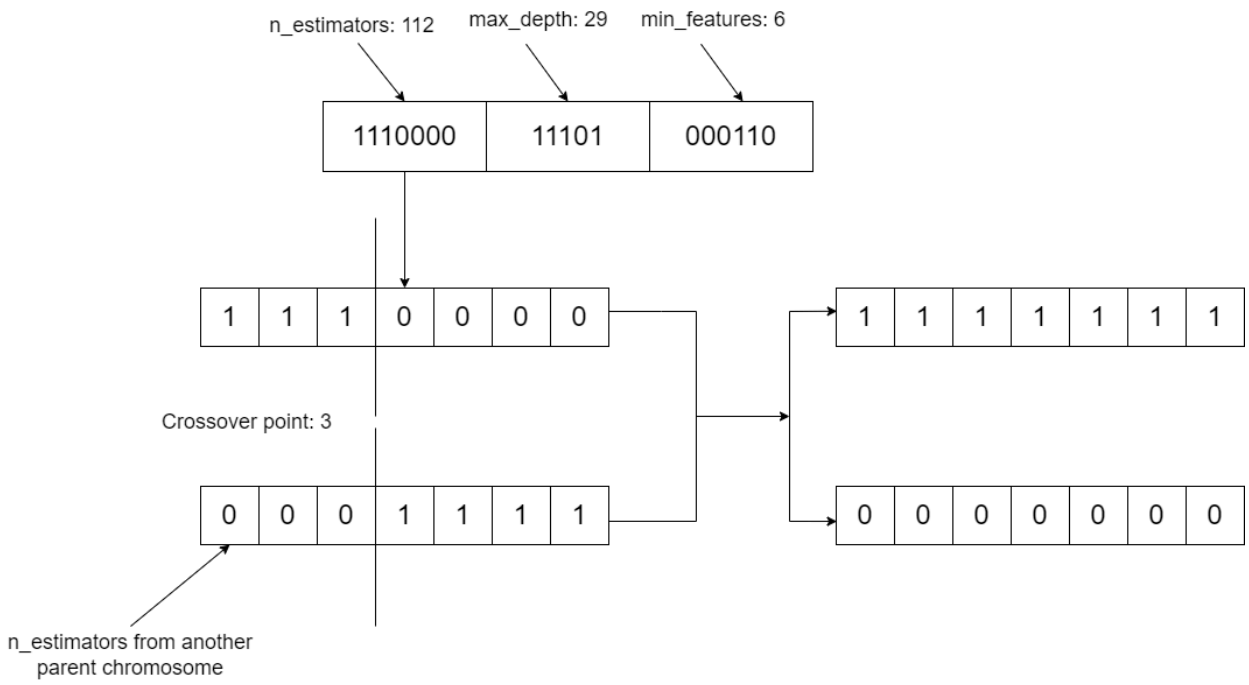


Figure 3: Single-point crossover for a single hyperparameter in a chromosome (Created with draw.io)

As seen from Figure 3, the crossover points are pre-determined to be 7 and 12, creating three hyperparameters (n_estimators, max_depth, and min_features). Parts of same hyperparameter from two parents is swapped using the single-point crossover operator to produce two offspring. In the figure, the n_estimators hyperparameter of two different parents (the entire chromosome of parent two is not shown) is crossed over to produce two new n_estimators hyperparameters.

5.f.v - Mutation operator

The mutation operator randomly flips the bit value at any particular position if the operator is triggered. This helps in preventing premature convergence, as it might help in escaping local minima or exploring different areas of the search space (Mitchell 8).

In this GA, the bit-flip mutation is used as it requires minimal computational power while effectively achieving its purpose. This process is better visualised in Figure 4:

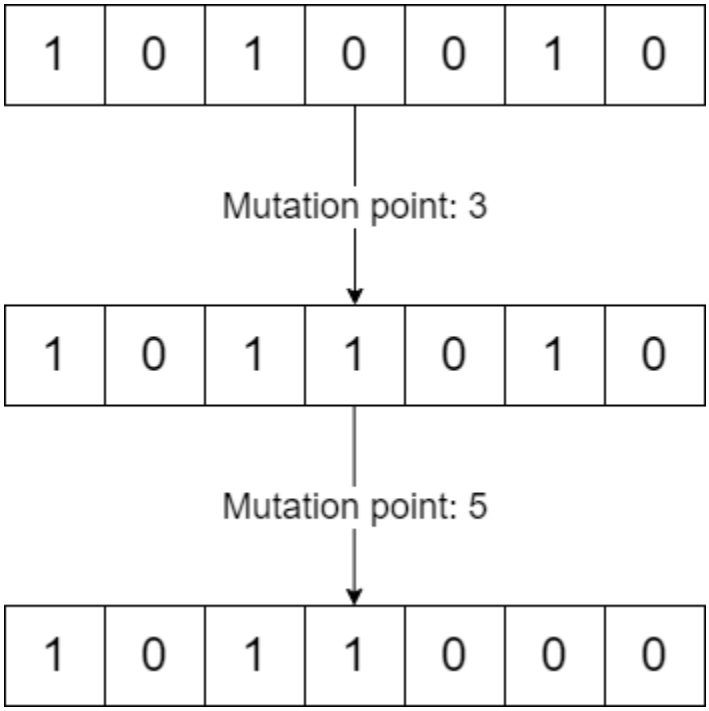


Figure 4: Two Mutation Operations on the same chromosome(Created with draw.io)

5.f.vi - Dynamic population scaling

A new concept incorporated into this GA, dynamic population scaling reduces the population size over successive generations by "killing off" the worst individuals in a generation. Inspired by the death of individuals in a population, the number of individuals killed is influenced by the decay rate. This mechanism is beneficial when the number of individuals to evaluate per generation is high, or if each solution (hyperparameter configuration) takes a significant amount

of time to train and test. This accelerates convergence, killing off poor-performing individuals who would have otherwise misguided the focus of the GA.

5.f.vii - GA Parameter Evaluation

De Jong's study (1975) discovered that a good GA performance requires a high crossover rate, a low mutation rate inversely proportional to the population size, and a moderate population size (Goldberg 71). The selection of parameters for this GA largely depends upon the size of the hyperparameter search space and the time available for the GA to find an optimal solution. With larger search spaces, a higher number of generation count is recommended. The recommended values for other parameters are as follows:

Table 5: Recommended parameter values for the GA

Parameter		**Value**	**Effect**
Crossover Rate		0.7	A higher crossover rate leads to higher divergence and vice-versa.
Mutation Rate		0.01	A higher mutation rate leads to a loss of useful information; a lower rate might lead to premature convergence.
Selection operator parameters	Alpha rank	0.4 - 0.6	A higher rate leads to a slower convergence and vice-versa.
	Beta Rank	1.4 - 1.6	A higher rate leads to faster

			convergence and vice-versa.
Elitism rate		0.02 - 0.05	Higher values lead to accelerated convergence and vice-versa.
Decay Rate		0 - 0.01	Higher values accelerate convergence and reduce computation time
Population Size		50 - 300	A higher population size is recommended for larger search spaces. A higher size requires more computational time to evaluate one generation.
Generation Count		25 - 100	A higher generation count can lead to better solutions, especially in a larger search space. A higher generation count can benefit from a lower selection pressure to evaluate all areas of the search space, but takes a significantly longer time to finish the entire GA process.

6 - Analysis and Evaluation

This data analysis aims to determine the effectiveness of the GA, BS and RS in optimising Gradient Boosting Regressor models for predicting housing prices. The objective function used to calculate the performance of each hyperparameter configuration was Mean Absolute Error. **The Roman numerals in Appendix 1 (Section 9) indicate the run number.**

6.a - Dataset Characteristics

The table below summarises the key characteristics of the two datasets used after pre-processing:

Table 6: Key Characteristics of both datasets used

Characteristic	California Dataset	Melbourne Dataset
No. of rows	20640	8895
No. of columns	7	360
Numerical Columns	7	9
Categorical columns	0	351
Skewness of Target Variable	0.9778	2.412
No. of Outliers	1071	420
Ratio of rows to columns	2948.57	24.71
Ratio of data points to outliers	19.27	21.18

By the traditional definition, the Melbourne dataset is a more complex dataset to analyse than the California dataset. As can be seen from Table 6, both datasets have the same ratio of data points to outliers. However, the ratio of rows to columns in the Melbourne dataset is far lower than the California dataset, making feature extraction more challenging. The data in the Melbourne dataset also shows a very strong positive skew (2.412), unlike the relatively balanced California dataset (0.9778).

Table 7: Average loss values of best solution found from optimisation algorithms for all settings (lower loss value represents a better solution)

	GA	BS	RS
Melbourne	148531.81183	158608.79783	156534.20781
California	0.44580	0.44832	0.44719

Table 8: Average improvement of GA over other optimisation algorithms

Dataset	Algorithm Comparison	Improvement of GA
Melbourne	GA vs BS	6.35%
	GA vs RS	5.11%
California	GA vs BS	0.56%
	GA vs RS	0.31%

As can be observed from Tables 7 and 8, the GA shows significant improvement in the Melbourne dataset over the California dataset. This could be attributed to the following factors:

- The outliers in the Melbourne dataset magnify the impact of skewness. Since a single surrogate model guides BS, it will likely be misguided. The advantage of GA not using hard-coded assumptions plays a crucial role here, as it takes input from multiple models and, thus, cannot be easily misguided.
- The Melbourne dataset's low row-to-column ratio makes feature and pattern extraction from input data more difficult. Therefore, finding hyperparameter configurations that can extract these features while not overfitting the model (as complex feature extraction often leads to overfitting) can prove to be challenging. Since BS only uses a few models to build and guide its surrogate model, these models are less likely to perform well, thus misguiding the surrogate model. This leads to overall less performance for the BS. However, in the California dataset, the BS performs similarly to GA and RS, performing better on a simpler dataset.
- GA and RS perform significantly better on the Melbourne dataset as they evaluate more models, thus increasing the possibility of finding a solution that extracts features while not overfitting the model. The use of logic in GA further improves the fitness of the solution, in comparison to RS and BS.

6.b - Small vs Large Search Spaces

Table 9: Average loss values for small and large search spaces for both datsets

(lower loss value represents a better solution)

Dataset		GA	BS	RS
Melbourne	Small	149862.78938	159144.16893	157979.32053
	Large	147200.83428	158073.42673	155089.09509
California	Small	0.44740	0.44959	0.45197
	Large	0.44420	0.44705	0.44241

Table 10: Average improvement of GA over other optimization algorithms

Dataset		Algorithm Comparison	Improvement of GA
Melbourne	Small	GA vs BS	5.83%
		GA vs RS	5.14%
	Large	GA vs BS	6.88%
		GA vs RS	5.08%
California	Small	GA vs BS	0.49%
		GA vs RS	1.01%
	Large	GA vs BS	0.64%
		GA vs RS	-0.40% (RS is better)

The keynotes from Tables 9 and 10 are summarised as follows:

- In all cases, the larger search space leads to better performance than the smaller search space (Table 9). This is due to the wider range of options for the hyperparameters, such as

n_estimators for which an increase in value to a certain extent almost always correlates to a decrease in the mean absolute error.

- For the large search space in the California dataset, RS provides the best average results, beating the GA by 0.40% and BS by 1.04%. Run 5 yielded the best hyperparameter configuration for the California dataset across all algorithms. However, RS did not result in better performance than the other two algorithms in the other three cases, highlighting the inconsistency of results in reliance on sole random number generation.
- It can be seen in both cases that GA shows a higher improvement over BS in the larger search space over the smaller search space, demonstrating that the curse of dimensionality does not affect GA as adversely as BS.

6.c - Evaluation based on Time Constraints

Time plays a crucial factor. Depending on the context, a more time-efficient algorithm could be used over an algorithm which finds a more accurate model. This section is divided into three subsections: short, moderate and extended timeframes.

6.c.i - Relative Short Timeframes

Table 11: Melbourne Dataset for Relative Short Timeframes (lower loss value represents a better solution)

Algorithm	Best Loss Achieved (Mean absolute error) - Small search space	Time taken for the best run (seconds)	Best Loss Achieved (Mean absolute error) - Large search space	Time taken for the best run (seconds)
GA	145072.974	8449.81	150437.045	19268.11
BS	158814.269	3637.05	159205.788	5758.87
RS	158015.580	9744.73	154185.399	30686.02

Table 12: California Dataset for Relative Short Timeframes (lower loss value represents a better solution)

Algorithm	Best Loss Achieved (Mean absolute error) - Small search space	Time taken for the best run (seconds)	Best Loss Achieved (Mean absolute error) - Large search space	Time taken for the best run (seconds)
GA	0.4524	12310.38	0.4398	33087.85
BS	0.4483	5620.66	0.4472	12539.72
RS	0.4521	14238.25	0.4435	28838.12

The data collected are as follows:

- Run 1 for small and Run 5 for large search space for GA
- Run 1 for small and Run 4 for large search space for BS and RS.

In the Melbourne dataset (Table 11), The GA achieves the lowest loss in both small and large search space but takes longer compared to BS. Despite GA's relatively mediocre performance in the California dataset (Table 12), it gives the best results in the large search space. RS and BS do not lag much behind.

As the dataset size increases, GA shows an even larger time overhead when compared to BS. BS's surrogate model and lower number of evaluations become advantageous as the complexity of evaluating a single model does not play a crucial role, whereas the same cannot be said about the GA and RS. This may point to scalability issues for the GA when time is considered.

However, if accuracy is the main factor, GA proves to be the best algorithm, especially when larger search spaces are considered. RS shows similar scalability issues.

6.c.ii - Relative Moderate Timeframes

Table 13: Melbourne Dataset for Relative Moderate Timeframes (lower loss value represents a better solution)

Algorithm	Best Loss Achieved (Mean absolute error) - Small search space	Time taken for the best run (seconds)	Best Loss Achieved (Mean absolute error) - Large search space	Time taken for the best run (seconds)
GA	150030.295	16860.30	147458.745	40307.5016
BS	159647.336	12428.45	157841.321	34036.84
RS	161036.422	19885.97	154028.578	59106.25

Table 14: California Dataset for Relative Moderate Timeframes (lower loss value represents a better solution)

Algorithm	Best Loss Achieved (Mean absolute error) - Small search space	Time taken for the best run (seconds)	Best Loss Achieved (Mean absolute error) - Large search space	Time taken for the best run (seconds)
GA	0.4479	25186.57	0.4513	54190.39
BS	0.4507	13810.1319	0.4469	41715.9442
RS	0.4546	27838.08	0.4378	62266.17

The data collected are as follows:

- Run 2 for small and Run 6 for large search space for GA
- Run 2 for small and Run 5 for large search space for BS and RS.

Similar to short timeframes, the GA performs exceedingly better in the Melbourne dataset (Table 13). In the California dataset (Table 14), the GA gives the best solution in the small search space. However, it has the worst solution in the large search space. When graph C from section 9.a.vi is looked at carefully, which shows the progression of fitness for large search space in the California dataset, the GA starts bad but still manages to improve the solution by 2.5%, highlighting the importance of a good initial population. This also underscores that RS and BS are more optimised for balanced rather than skewed datasets. BS remains the most computationally efficient.

6.c.iii - Relative Extended Timeframes

Table 15: Melbourne Dataset for Relative Extended Timeframes (lower loss value represents a better solution)

Algorithm	Best Loss Achieved (Mean absolute error) - Small search space	Time taken for the best run (seconds)	Best Loss Achieved (Mean absolute error) - Large search space	Time taken for the best run (seconds)
GA	152173.944	25314.84	145453.774	64975.61
BS	158970.901	34890.40	157173.170	68008.81
RS	154885.959	38597.93	157053.308	98603.70

Table 16: California Dataset for Relative Extended Timeframes (lower loss value represents a better solution)

Algorithm	Best Loss Achieved (Mean absolute error) - Small search space	Time taken for the best run (seconds)	Best Loss Achieved (Mean absolute error) - Large search space	Time taken for the best run (seconds)
GA	0.4446	40564.34	0.4428	95751.81
BS	0.4498	37037.58	0.4471	60774.22
RS	0.4493	55975.07	0.4459	124947.41

The data collected are as follows:

- Average of Run 3 and 4 for small and Run 7 and 8 for large search space for GA (time is also averaged)
- Run 3 for small and Run 6 for large search space for BS and RS

GA maintains the same lead in the Melbourne dataset. However, it returns the best result for both cases in the California dataset, though the improvement isn't significant. BS remains the most computationally efficient algorithm for all cases, and RS remains the worst.

6.d - Drawbacks of each algorithm

6.d.i - Genetic Algorithm

GA is highly dependent on the randomly generated first generation, much like BS's initial acquisition function. It shows an increase of 1.5%-5% in the performance of the best solution in the final generation to the best solution in the initial generation, no matter how good the initial

generation. For instance, in graph A under section 9.a.i, the solution in the initial generation was better than most of the final solutions found by the other configurations, yet the GA was able to further improve it by an additional 1.85%. This underscores the importance of a good quality initial generation, which can be solved by populating a part of the population with solutions from diverse areas of the search space previously known to have worked well.

Another key drawback of this GA is its longer evaluation time, especially when the search space is large, due to the higher number of generations and population size required to find an optimal solution. The inclusion of wider hyperparameter ranges might also take longer to evaluate. For example, evaluating the hyperparameter *n_estimators* at a value of 600 requires significantly more time than evaluating it at 300.

6.d.ii - Bayes Search

From the data collected, it can be seen that the surrogate model fails to predict hyperparameter configurations that improve performance after a certain point. In graph A under section 9.b.i, the best solution found showed minimal improvement from the solution found in iteration 17, even though the whole run was for 150 iterations. In graph B, the model flat lines after 90 iterations. This indicates that BS struggles to identify hyperparameter sets that yield further performance improvements after a certain point.

BS also relies heavily on Cross-Validation. Cross-Validation divides the dataset into equal parts and trains the model on each split of the dataset. This helps in getting a more accurate loss value for the hyperparameter configuration, which is important considering it does not evaluate many

models and affects the surrogate model and acquisition function significantly. However, this increases computational time, which is mitigated by BS's overall efficiency, as it evaluates fewer models and remains significantly more computationally efficient than GA or RS.

6.d.iii - Random Search

RS takes longer to converge as it frequently selects suboptimal solutions that are both time-consuming to evaluate and perform worse compared to GA or BS. One such example can be taken for the hyperparameter max_depth. Not only does a higher depth take a significantly longer time to evaluate but also leads to overfitting. Overfitted models are not passed in the GA or BS, so they don't evaluate these models as often as RS. This could be attributed to RS's significantly longer evaluation times in comparison to GA and BS.

7 - Limitations and conclusion

7.a - Limitations of this Research for further improvements

The implementation of the GA was pretty basic and can implement advanced operators such as dominance, diploidy, etc. for possible improved performance. Parallelisation could also be implemented to evaluate the solutions faster. It can also be made into a multi-objective optimisation algorithm, taking in various factors such as time to train and get results back from the model, accuracy, and precision for both the training and testing dataset so that the model performs better in real-world conditions. This flexibility is something that is lacking in currently used optimisation algorithms. The relationship between GA's parameters, such as elitism,

reduction, crossover rate, and mutation rate, can be further examined to establish a connection between them and the GA's convergence behaviour and solution quality.

7.b - Conclusion

This study underscores the potential of GAs as an optimization tool, particularly for complex datasets like Melbourne, where traditional methods like BS and RS struggled with skewness and outliers. While it still cannot beat BS in terms of efficiency, it performs more consistently, from complex datasets to larger search spaces. The preferred algorithm ultimately depends upon the priorities of the optimization problem. If efficiency is prioritised, BS is easily the best choice. However, if accuracy, robustness and consistent performance are the objectives, GAs prove to be the more suitable choice.

This research can also be extended to different problem domains such as financial, climate, health data, and many more applications where AI models are currently being used. This GA can be further used to optimise other categories of models, from simple Linear or Tree-based models to complex Neural Networks. Its adaptability in finding models for different types of datasets which all have different requirements makes it a valuable tool for hyperparameter optimisation.

8. Works Cited

David, Davis. “Hyperparameter Optimization Techniques.” *freeCodeCamp*, 12 October 2020, https://www.freecodecamp.org/news/hyperparameter-optimization-techniques-machine-learning/. Accessed 10 January 2025.

Frazier, Peter I. “A Tutorial on Bayesian Optimization.” 2018. *arxiv*, https://arxiv.org/pdf/1807.02811. Accessed 21 July 2024.

Goldberg, David Edward. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.

“Gradient Boosting in ML.” *GeeksForGeeks*, 31 March 2023, https://www.geeksforgeeks.org/ml-gradient-boosting/. Accessed 11 January 2025.

“GradientBoostingRegressor — scikit-learn 1.7.dev0 documentation.” *Scikit-learn*, https://scikit-learn.org/dev/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html. Accessed 13 January 2025.

Grefenstette, John. “Rank-based Selection.” *Handbook of Evolutionary Computation*, 1997, pp. 1 - 2. *C2.4 Rank-based selection - John Grefenstette*, https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=b61cec42e5aa997a20d563b2886c083b3e0d335c. Accessed 29 July 2024.

Koehrsen, Will. “Intro to Model Tuning: Grid and Random Search.” *Kaggle*, 2018, https://www.kaggle.com/code/willkoehrsen/intro-to-model-tuning-grid-and-random-search#Random-Search. Accessed 20 January 2025.

Mitchell, Melanie. *An introduction to genetic algorithms*. Fifth ed., Bradford Books, 1999.

Pandian, Shanthababu. “A Comprehensive Guide on Hyperparameter Tuning and its Techniques.” *Analytics Vidhya*,

https://www.analyticsvidhya.com/blog/2022/02/a-comprehensive-guide-on-hyperparameter-tuning-and-its-techniques/. Accessed 27 April 2024.

Pino, Tony. “Melbourne Housing Market.” *Kaggle*, https://www.kaggle.com/datasets/anthonypino/melbourne-housing-market. Accessed 30 April 2024.

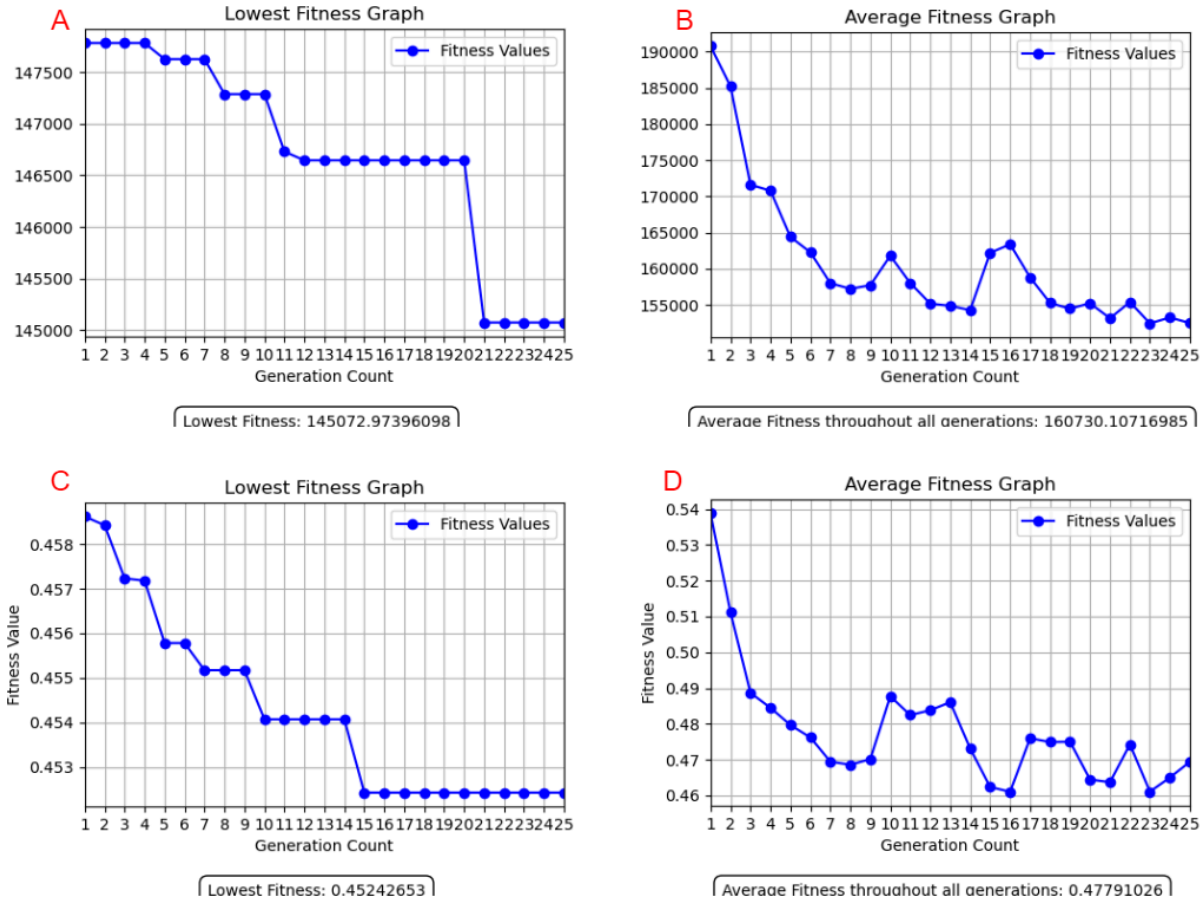
Theobald, Oliver. *Machine Learning for Absolute Beginners: A Plain English Introduction (Third Edition)*. Scatterplot Press, 2021.

Yang, Li, and Abdallah Shami. *On Hyperparameter Optimization of Machine Learning Algorithms: Theory and Practice*, 2022, pp. 4 - 33.

9. Appendix 1

9.a - Primary data collected from different configurations of the GA

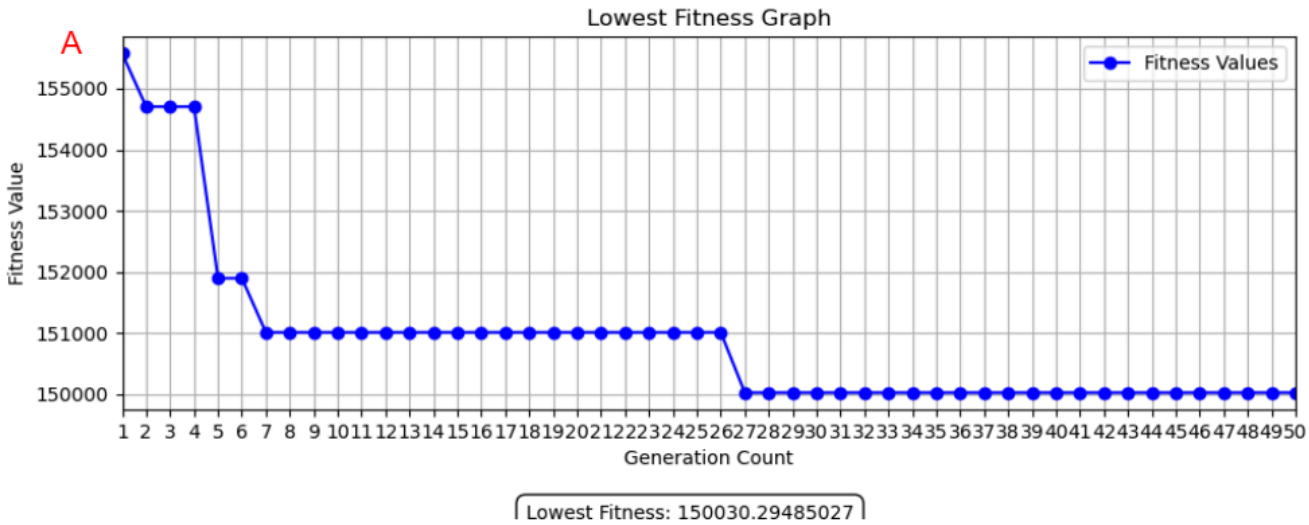
9.a.i - Low generation count and population size; high convergence rate; small search space

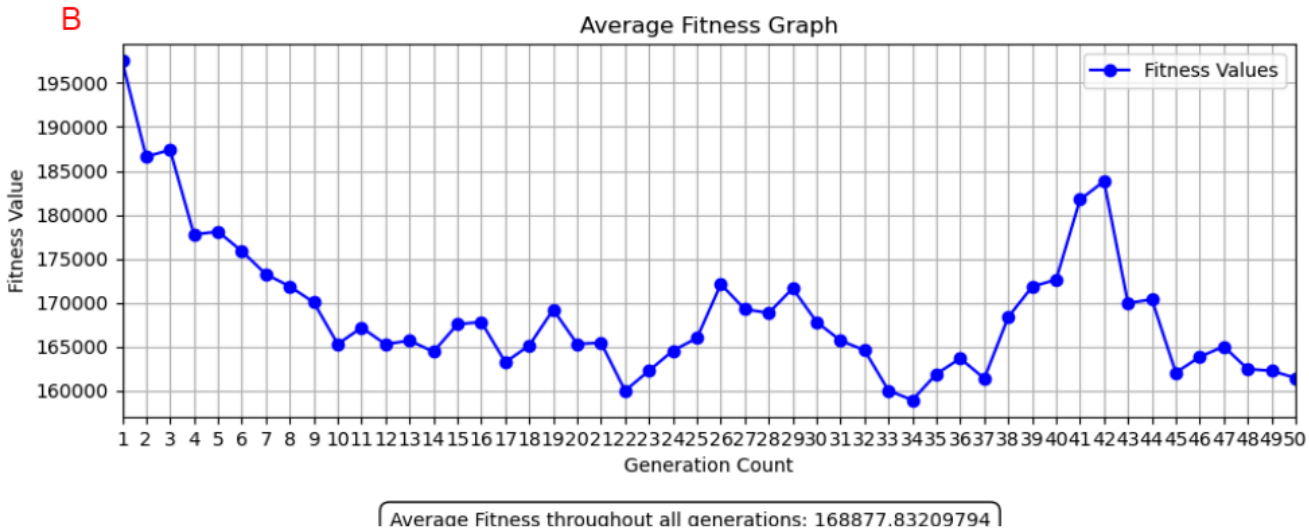


Population Size	60
Generation Count	25
Alpha Rank	0.4
Beta Rank	1.6
Elitism Rate	0.05
Reduction Rate	0.0
Relative Lowest Fitness (Melbourne)	0.99246
Relative Lowest Fitness (California)	0.96761
Time Taken (Melbourne) (seconds)	8449.8147
Time Taken (California) (seconds)	12310.3802

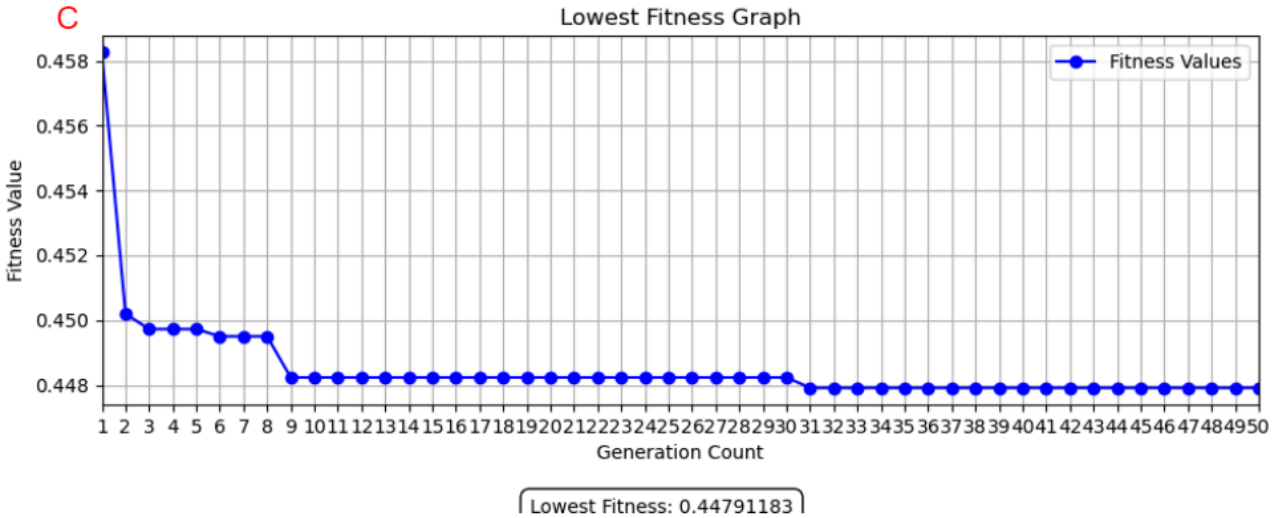
9.a.ii - Low population size and convergence rate; high maximum generation count; small search space

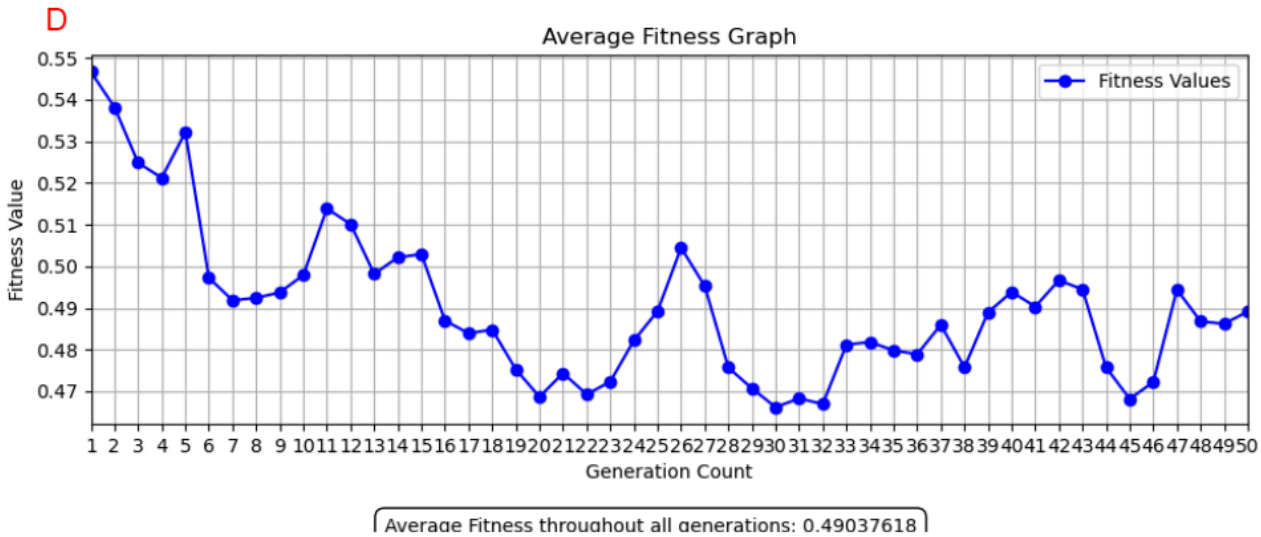
Note: The first two images are from the Melbourne dataset, and the second two are from the California dataset





The above two are from the data collected from the Melbourne dataset

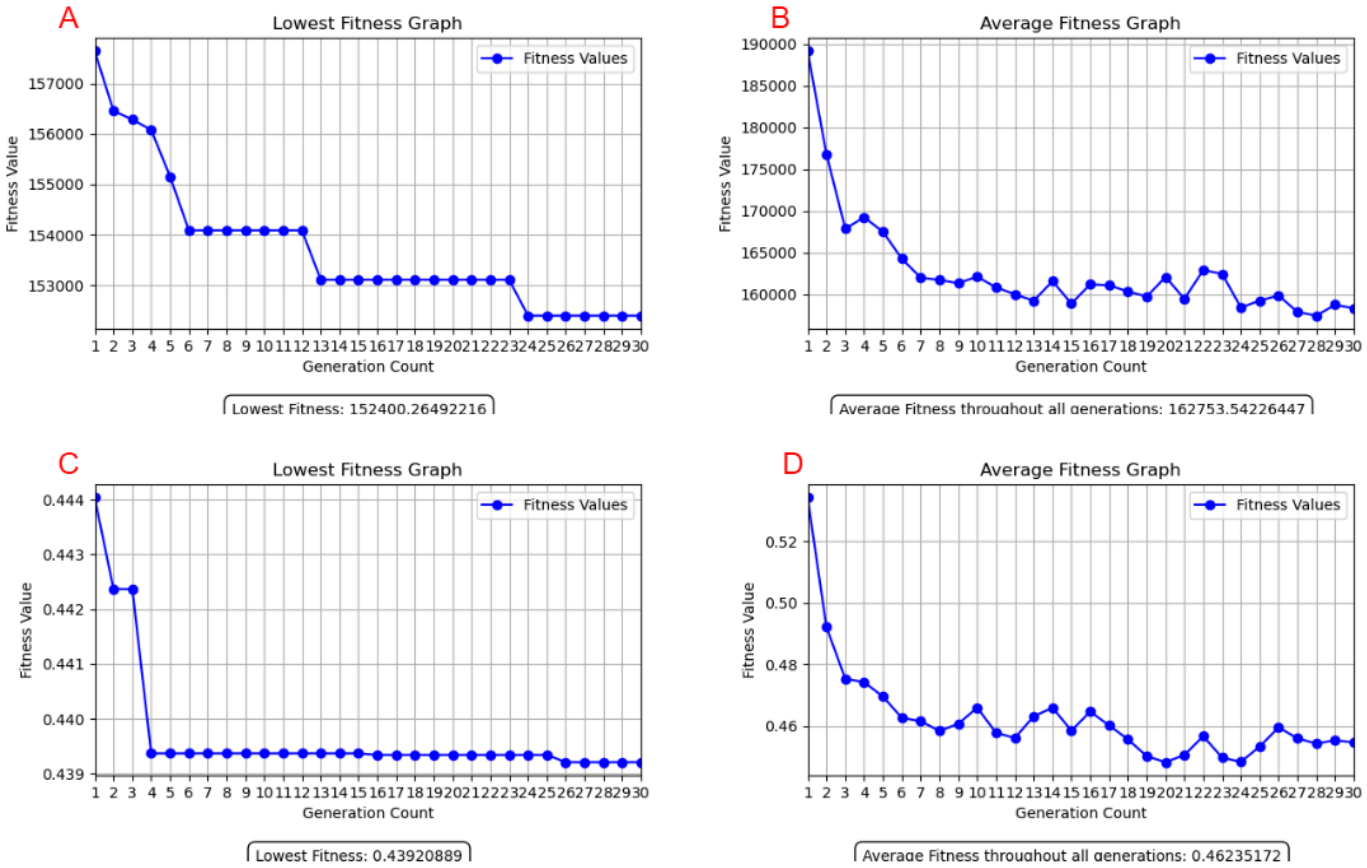




The above are taken from California dataset

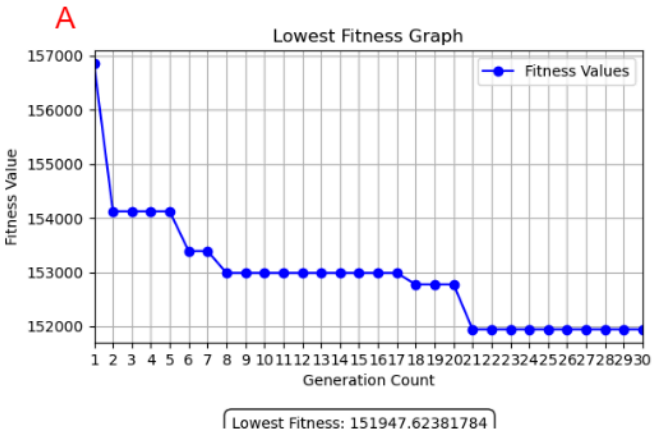
Population Size	60
Generation Count	50
Alpha Rank	0.8
Beta Rank	1.2
Elitism Rate	0.05
Reduction Rate	0.0
Relative Lowest Fitness (Melbourne)	0.95967
Relative Lowest Fitness (California)	0.97736
Time Taken (Melbourne) (seconds)	16860.3014
Time Taken (California) (seconds)	25186.5731

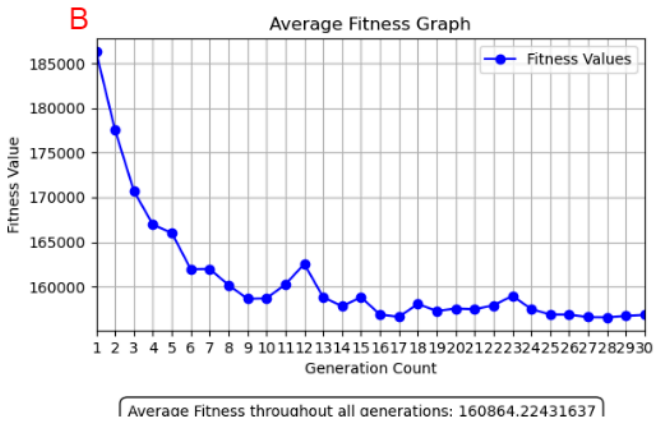
9.a.iii - High population size and convergence rate; low maximum generation count; low reduction rate; small search space
Note: The first row is from the Melbourne dataset, and the second row is from the California dataset

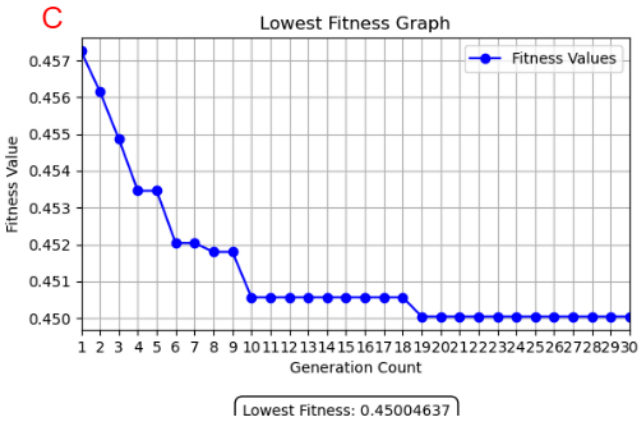


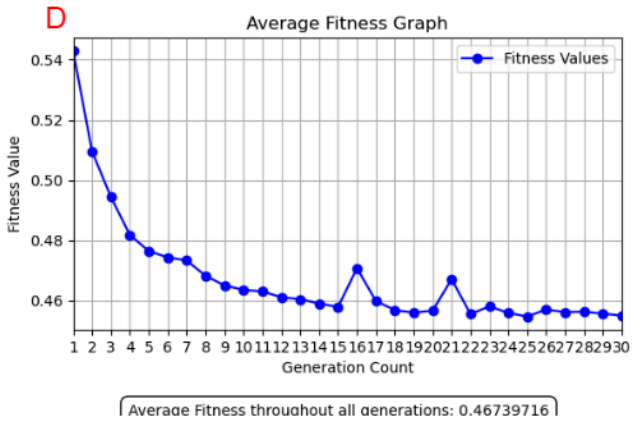
Population Size	200
Generation Count	30
Alpha Rank	0.4
Beta Rank	1.6
Elitism Rate	0.03
Reduction Rate	0.01
Relative Lowest Fitness (Melbourne)	0.94474
Relative Lowest Fitness (California)	0.99673
Time Taken (Melbourne) (seconds)	27542.0744
Time Taken (California) (seconds)	45387.9725

9.a.iv- High population size and convergence rate; low maximum generation count; high reduction rate; small search space
Note: The first row is from the Melbourne dataset, and the second row is from the California dataset





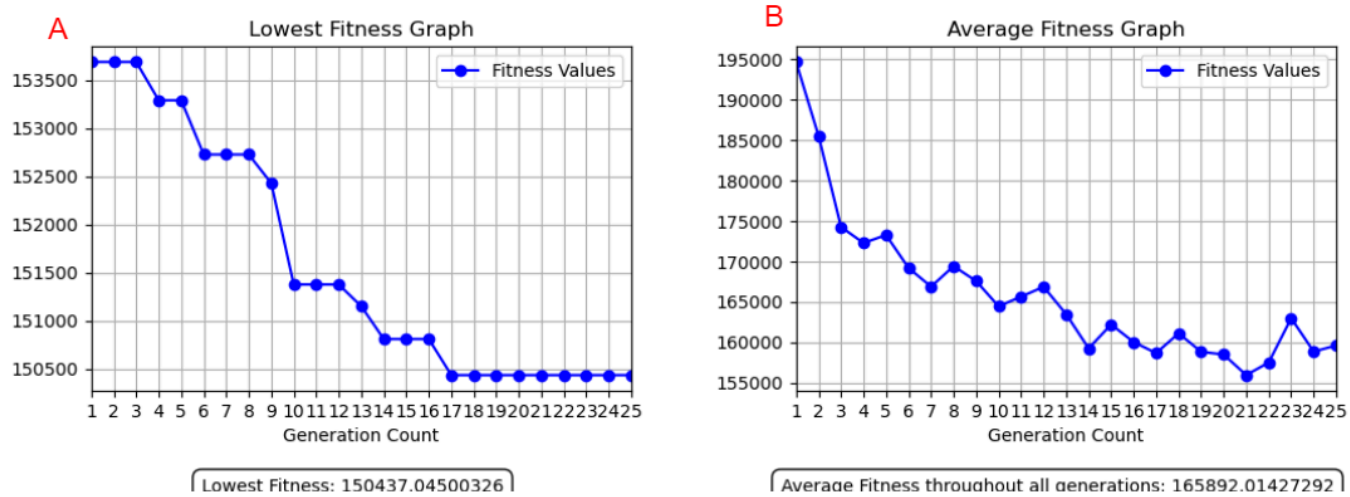


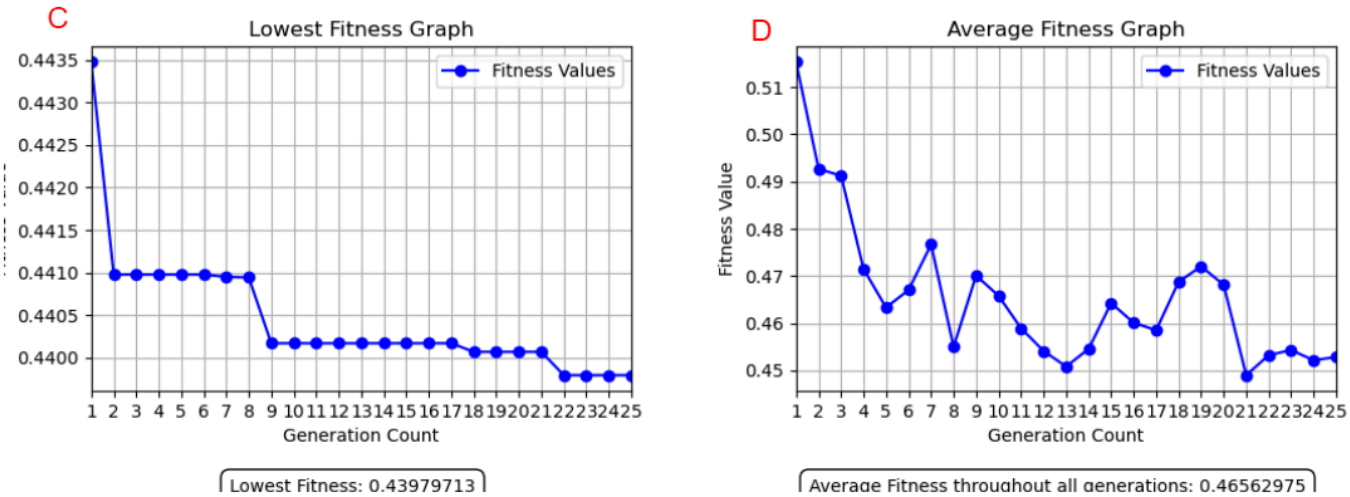


Population Size	200
Generation Count	30
Alpha Rank	0.4
Beta Rank	1.6
Elitism Rate	0.03
Reduction Rate	0.02
Relative Lowest Fitness (Melbourne)	0.94756
Relative Lowest Fitness (California)	0.97273
Time Taken (Melbourne) (seconds)	23087.5962
Time Taken (California) (seconds)	35740.7008

9.a.v - Low generation count and population size; high convergence rate; large search space

Note: The first row is from the Melbourne dataset, and the second row is from the California dataset

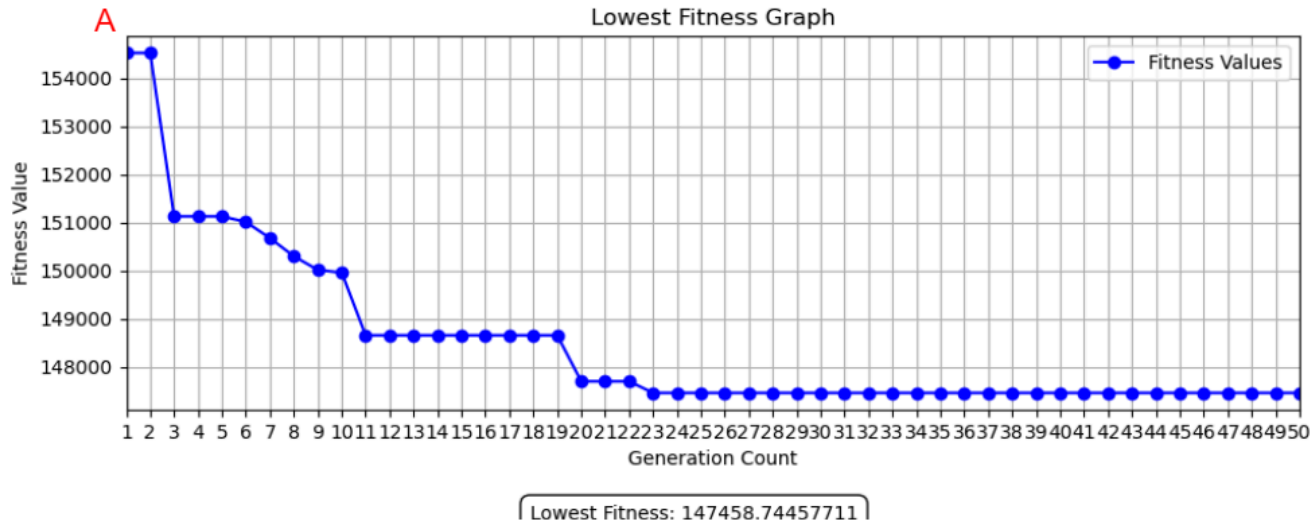


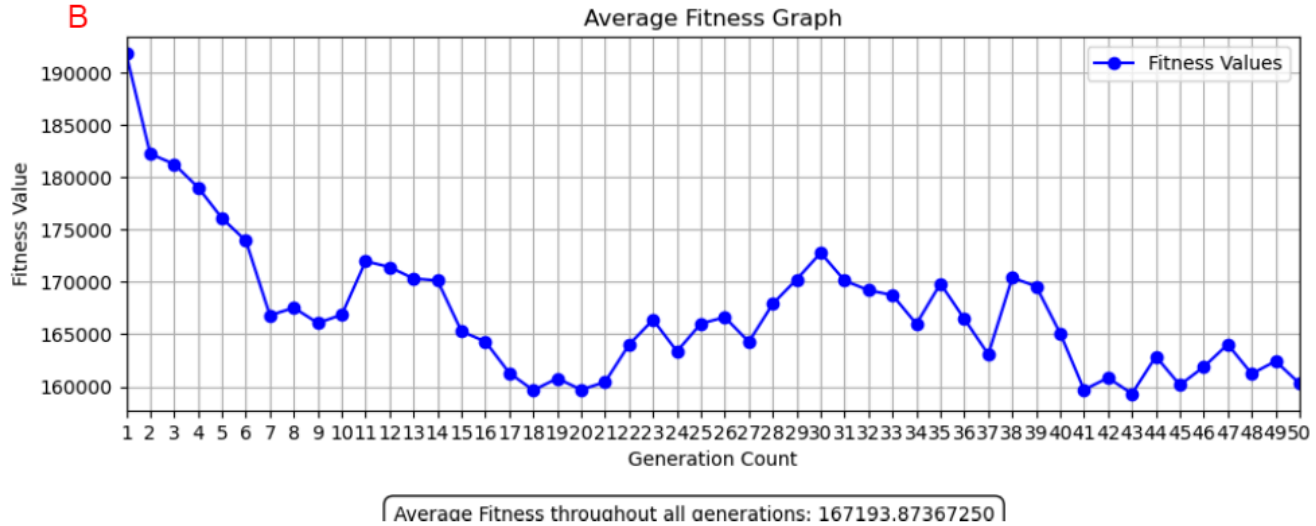


Population Size	60
Generation Count	25
Alpha Rank	0.4
Beta Rank	1.6
Elitism Rate	0.05
Reduction Rate	0.0
Relative Lowest Fitness (Melbourne)	0.95708
Relative Lowest Fitness (California)	0.99540
Time Taken (Melbourne) (seconds)	19268.1133
Time Taken (California) (seconds)	33087.8506

9.a.vi - Low population size and convergence rate; high maximum generation count; large search space

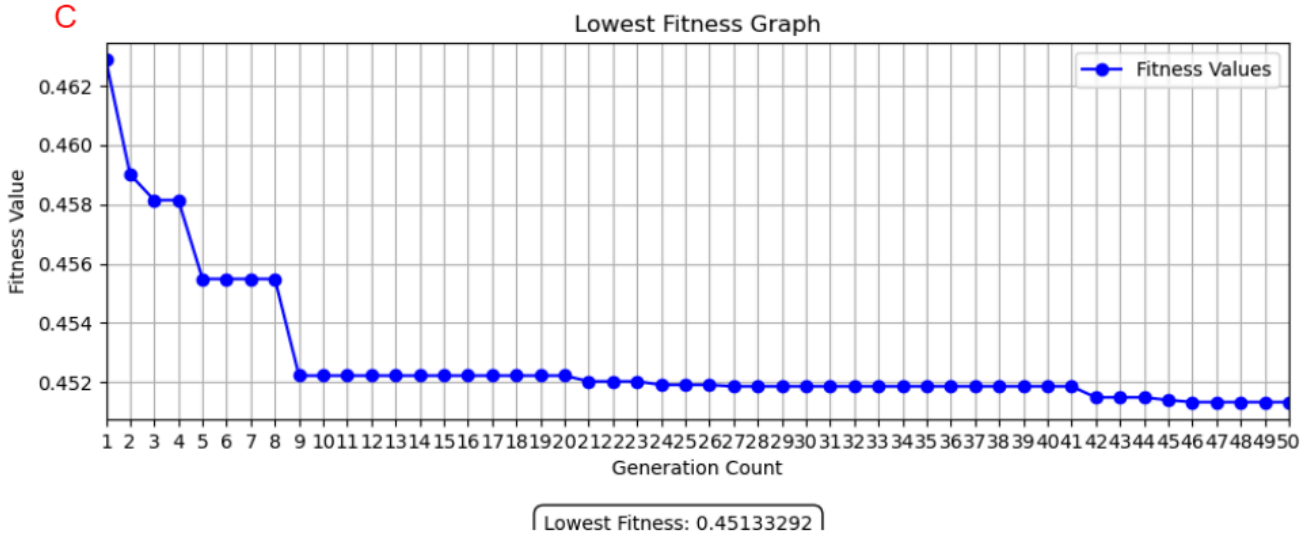
Note: The first two images are from the Melbourne dataset, and the second two are from the California dataset





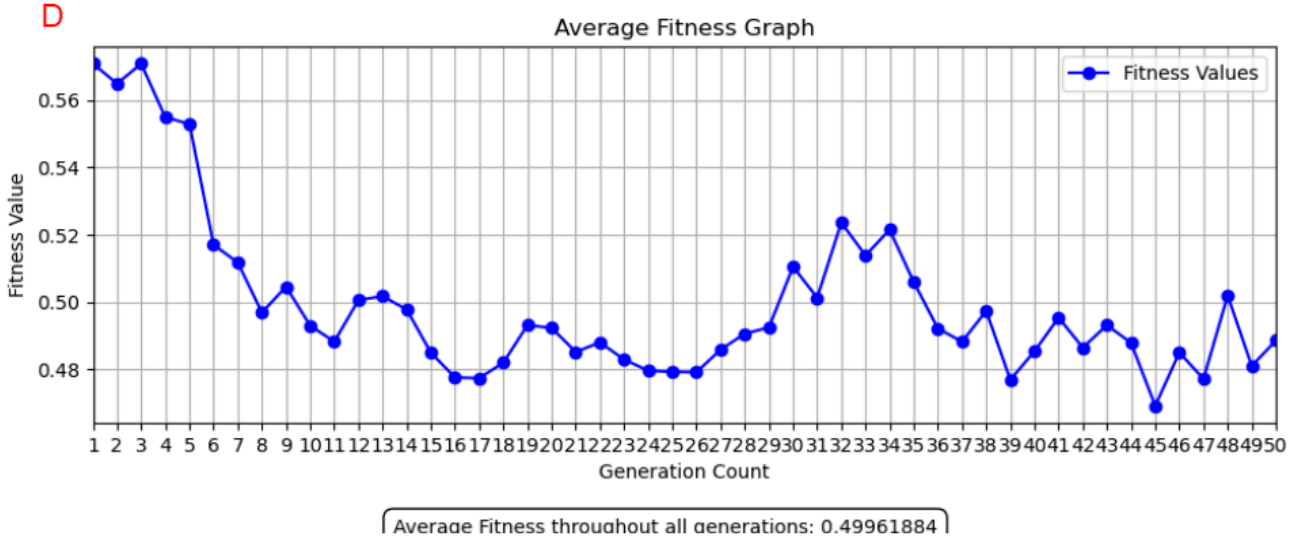
The above two are from the data collected from the Melbourne dataset

C



Lowest Fitness: 0.45133292

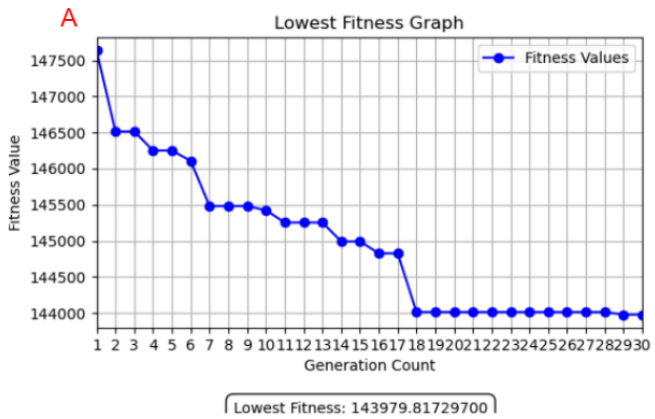
D

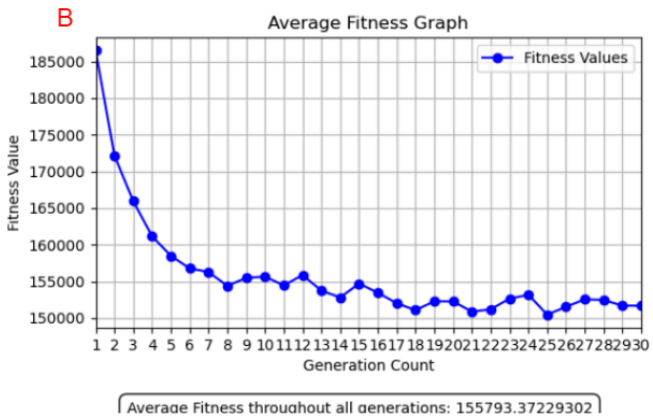


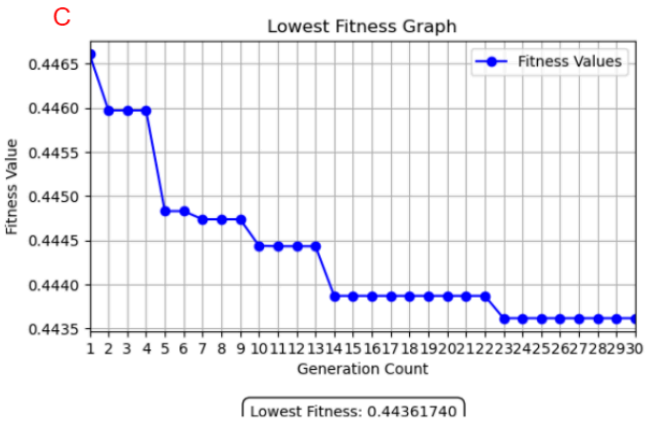
Average Fitness throughout all generations: 0.49961884

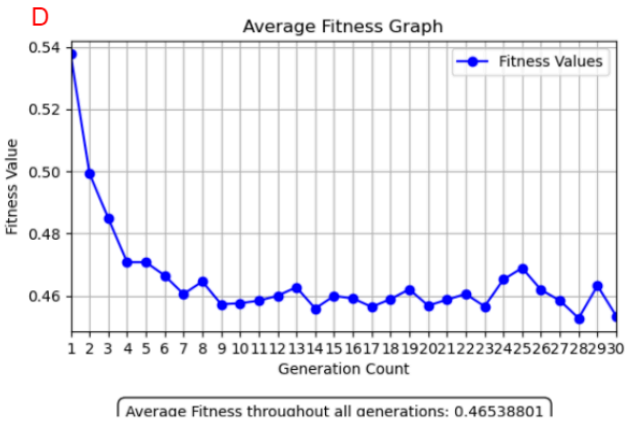
Population Size	60
Generation Count	50
Alpha Rank	0.8
Beta Rank	1.2
Elitism Rate	0.05
Reduction Rate	0.0
Relative Lowest Fitness (Melbourne)	0.97641
Relative Lowest Fitness (California)	0.96995
Time Taken (Melbourne) (seconds)	40307.5016
Time Taken (California) (seconds)	54190.3923

9.a.vii- High population size and convergence rate; low maximum generation count; low reduction rate; large search space
Note: The first row is from the Melbourne dataset, and the second row is from the California dataset





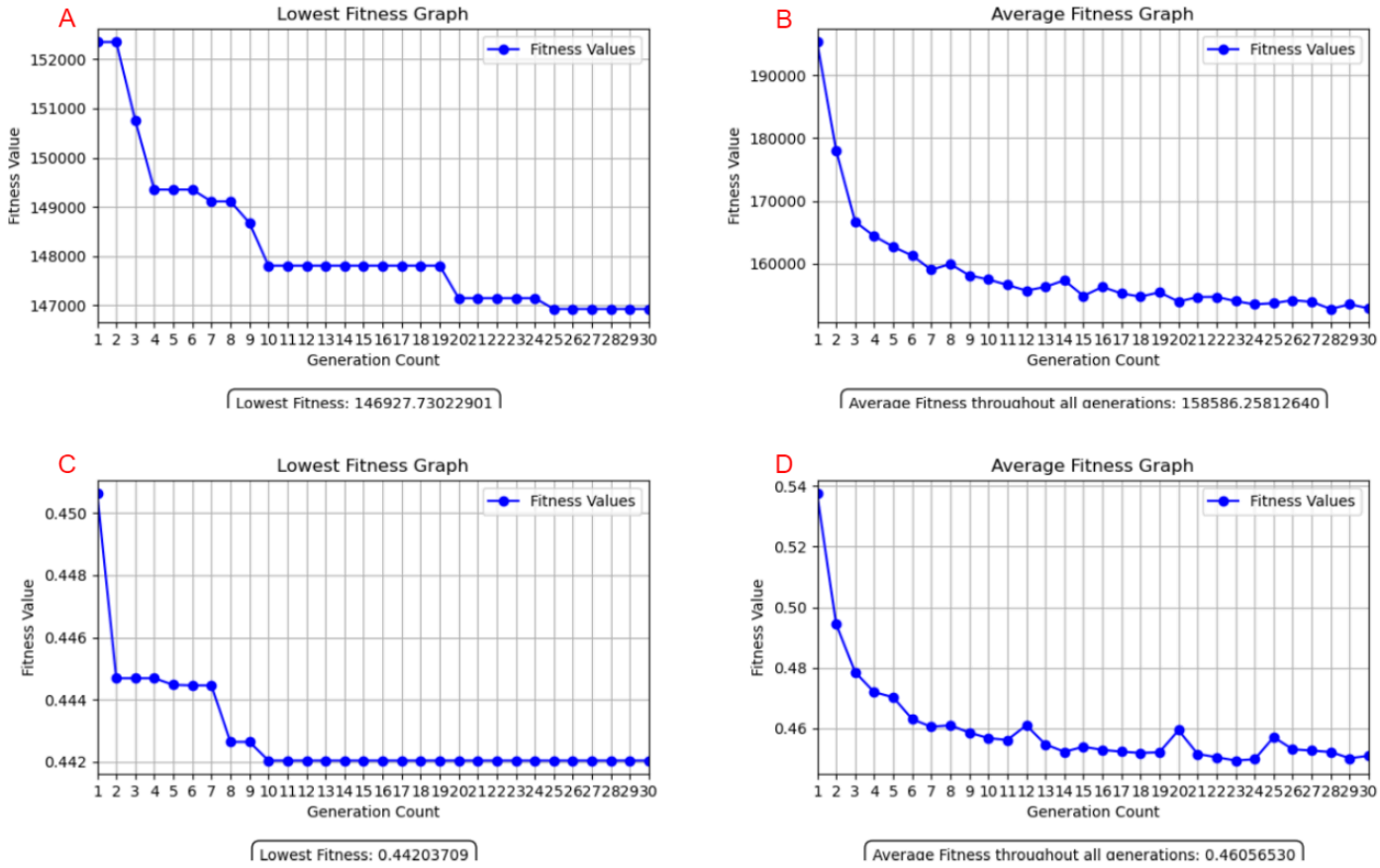




Population Size	200
Generation Count	30
Alpha Rank	0.4
Beta Rank	1.6
Elitism Rate	0.03
Reduction Rate	0.01
Relative Lowest Fitness (Melbourne)	1
Relative Lowest Fitness (California)	0.98682
Time Taken (Melbourne) (seconds)	71475.5145
Time Taken (California) (seconds)	103135.4315

9.a.viii- High population size and convergence rate; low maximum generation count; high reduction rate; large search space

Note: The first row is from the Melbourne dataset, and the second row is from the California dataset



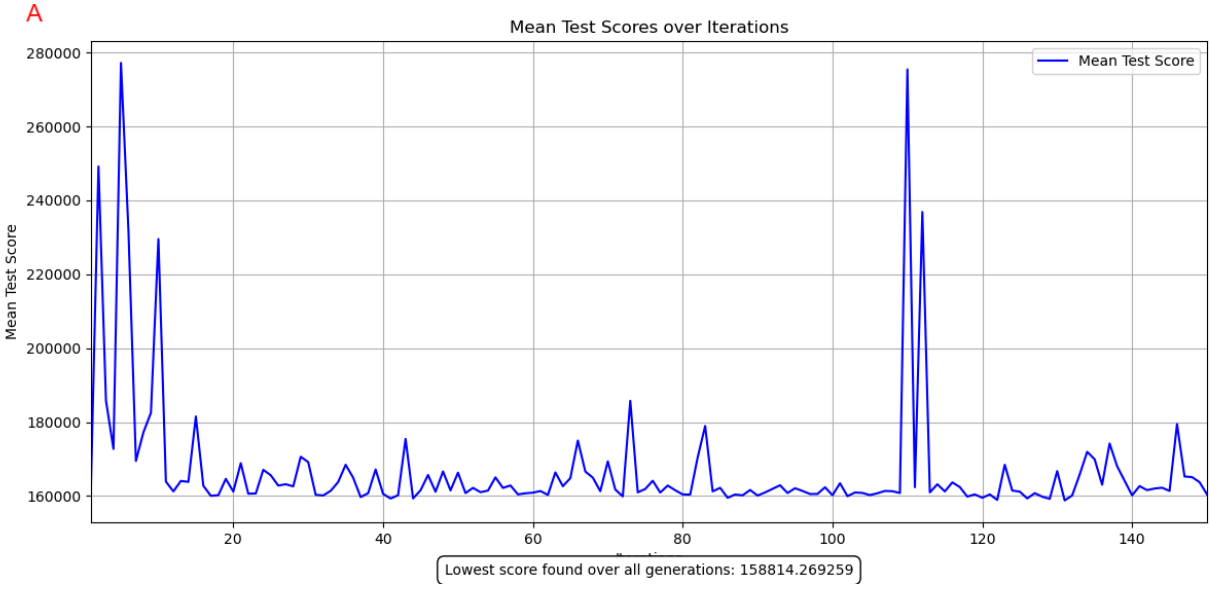
Population Size	200
Generation Count	30
Alpha Rank	0.4
Beta Rank	1.6
Elitism Rate	0.03
Reduction Rate	0.02
Relative Lowest Fitness (Melbourne)	0.97994
Relative Lowest Fitness (California)	0.99035
Time Taken (Melbourne) (seconds)	58475.7007
Time Taken (California) (seconds)	3

9.b - Primary data collected from different configurations of BS

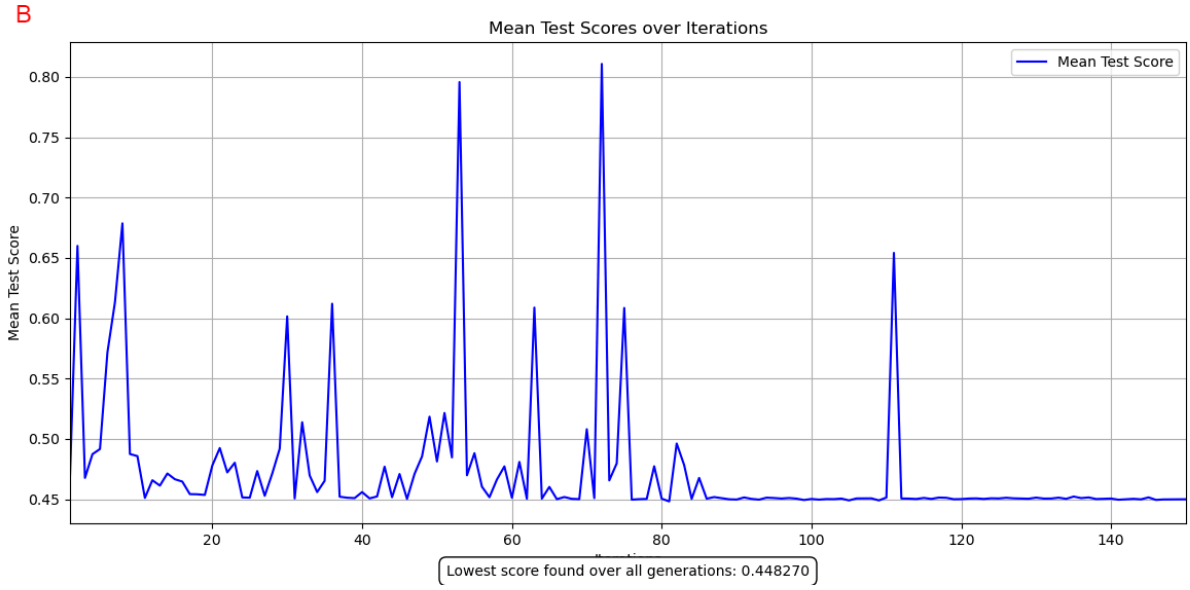
9.b.i - Low number of iterations; small search space

Note: The first image is taken from the Melbourne dataset, and the second is taken from the California dataset

A



B

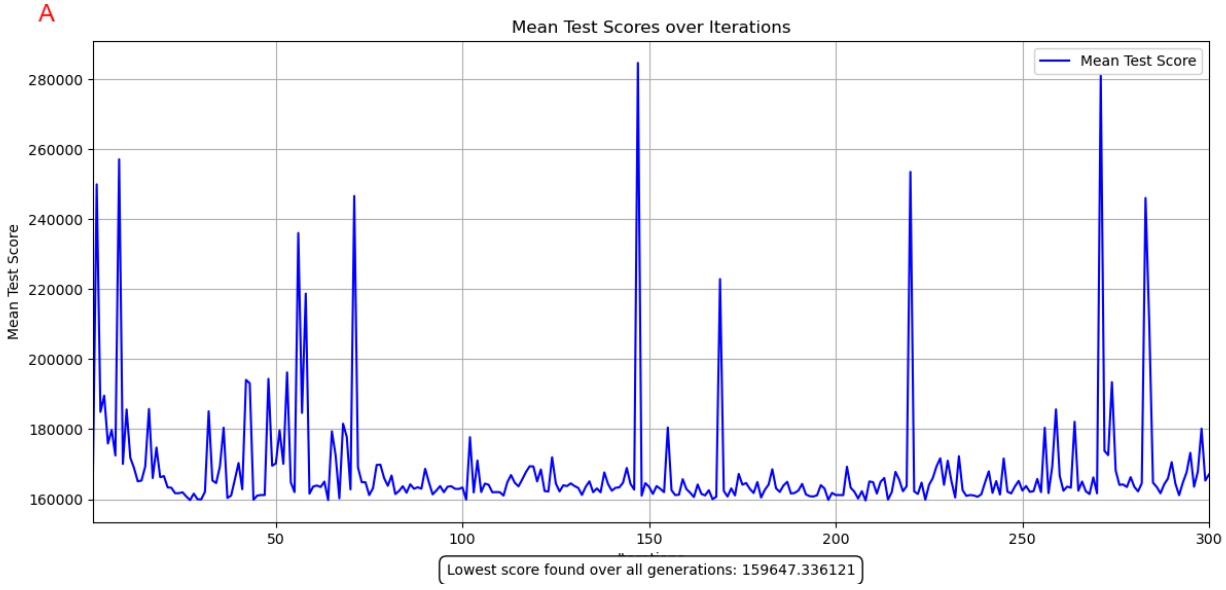


n_iter	150
Relative Lowest Fitness (Melbourne)	0.90659
Relative Lowest Fitness (California)	0.97658
Time Taken (Melbourne) (seconds)	3637.0532
Time Taken (California) (seconds)	5620.6578

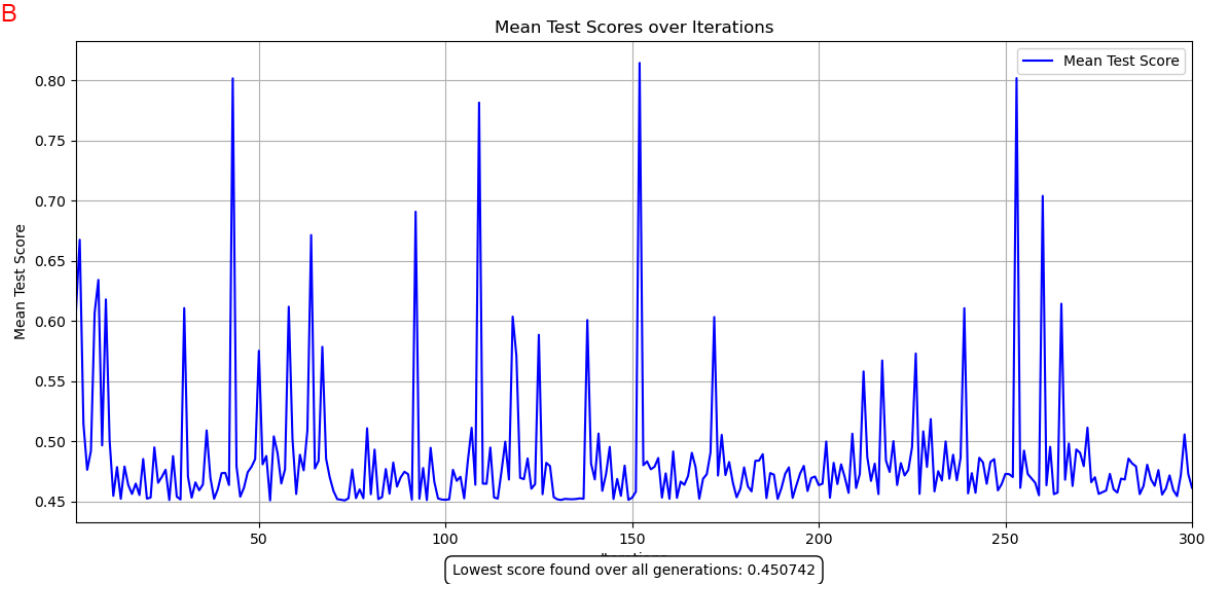
9.b.ii - Medium number of iterations; small search space

Note: The first image is taken from the Melbourne dataset and the second is taken from the California dataset

A



B

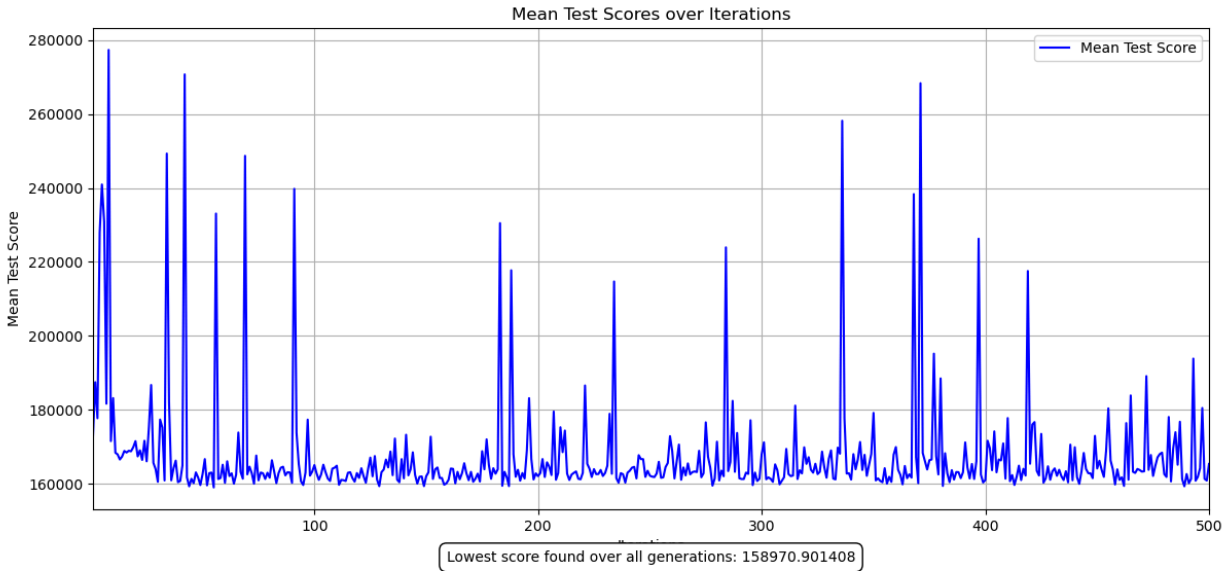


n_iter	300
Relative Lowest Fitness (Melbourne)	0.90186
Relative Lowest Fitness (California)	0.97123
Time Taken (Melbourne) (seconds)	12428.4459
Time Taken (California) (seconds)	13810.1319

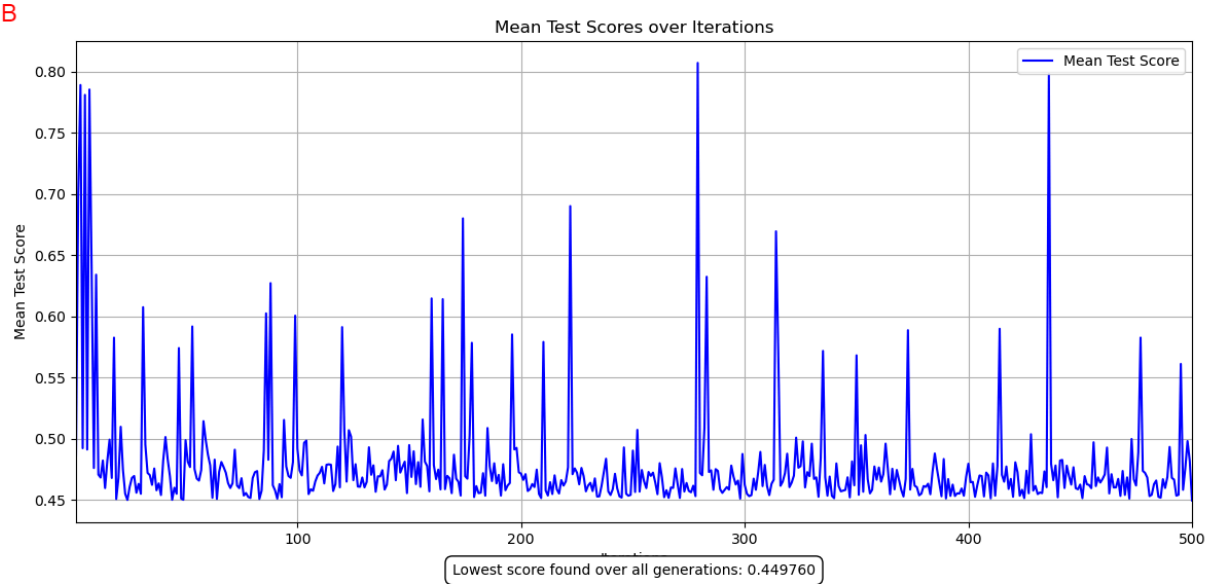
9.b.iii - High number of iterations; small search space

Note: The first image is taken from the Melbourne dataset and the second is taken from the California dataset

A



B

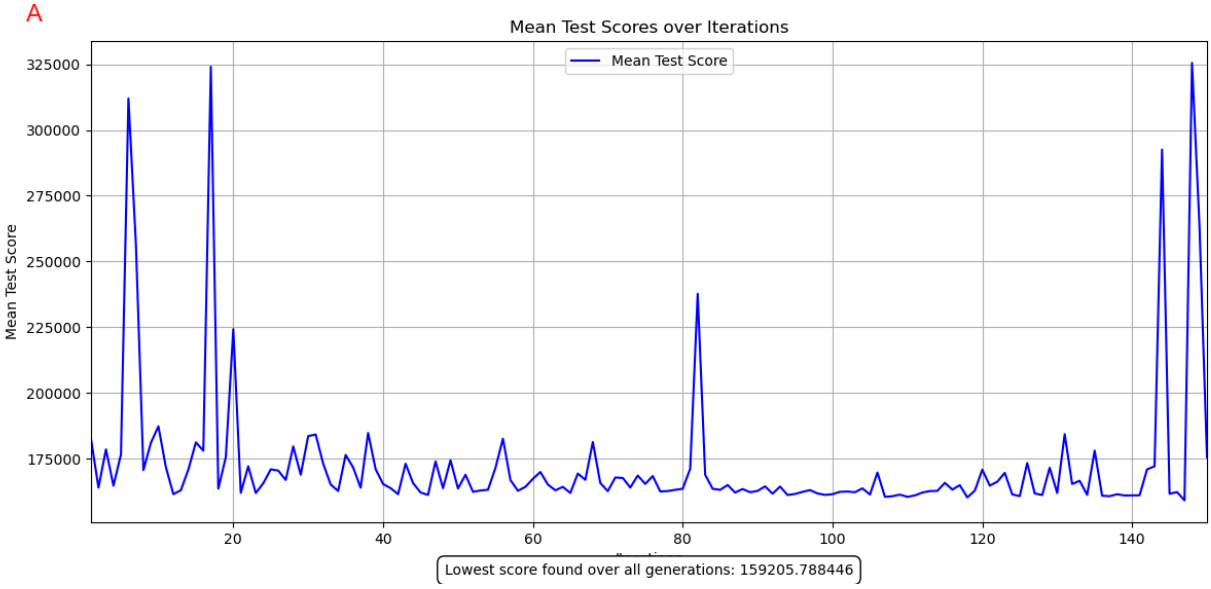


n_iter	500
Relative Lowest Fitness (Melbourne)	0.90570
Relative Lowest Fitness (California)	0.97335
Time Taken (Melbourne) (seconds)	34890.4003
Time Taken (California) (seconds)	37037.5801

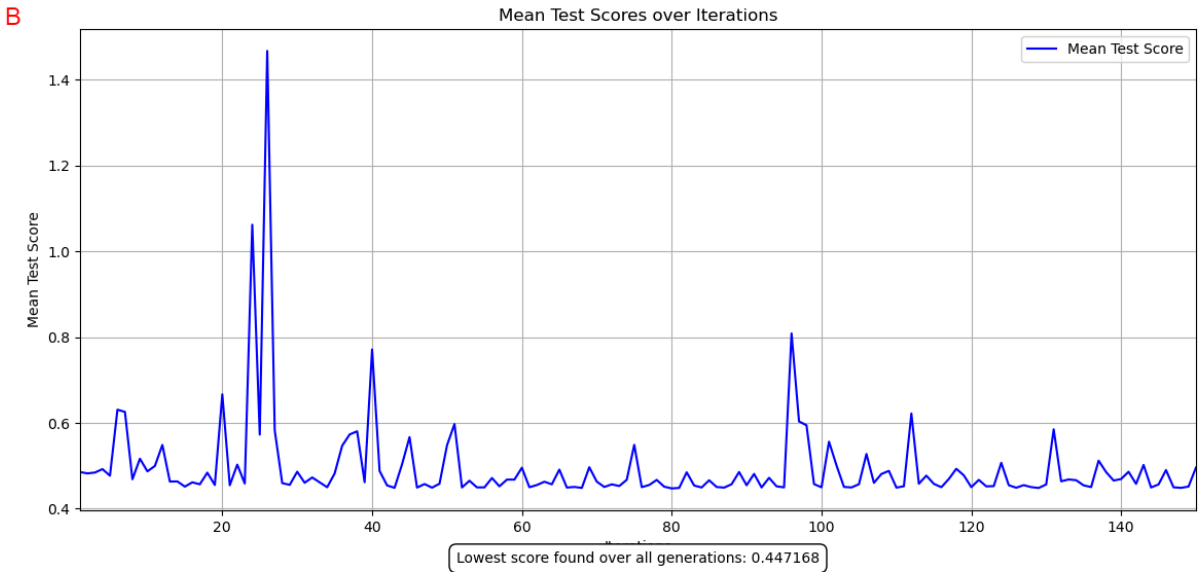
9.b.iv - Low number of iterations; large search space

Note: The first image is taken from the Melbourne dataset and the second is taken from the California dataset

A



B

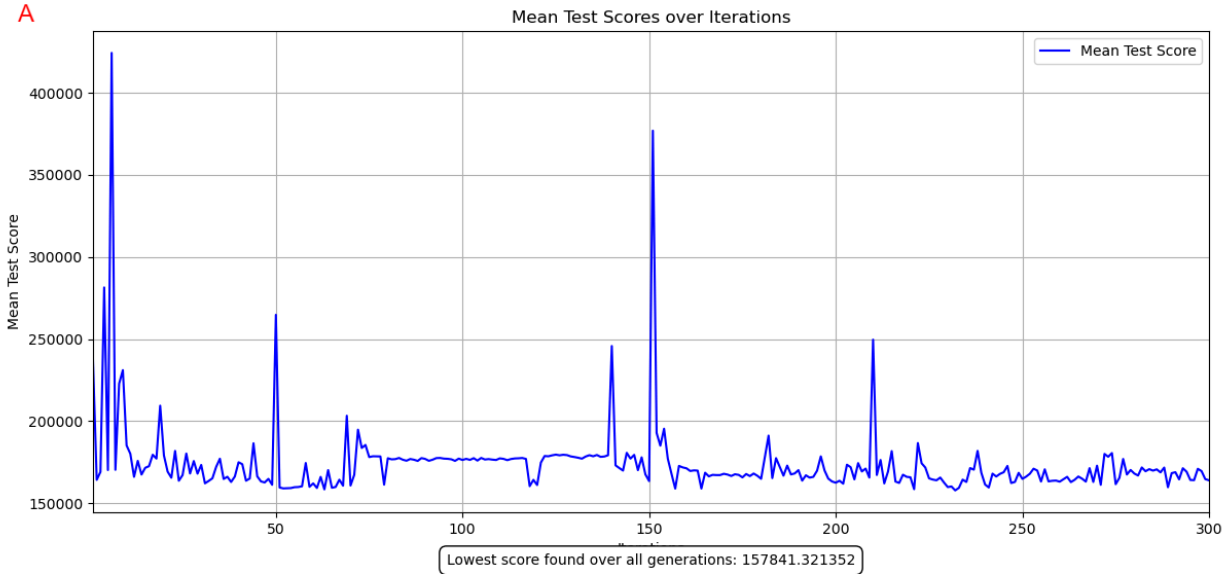


n_iter	150
Relative Lowest Fitness (Melbourne)	0.90436
Relative Lowest Fitness (California)	0.97899
Time Taken (Melbourne) (seconds)	5758.8686
Time Taken (California) (seconds)	12539.7213

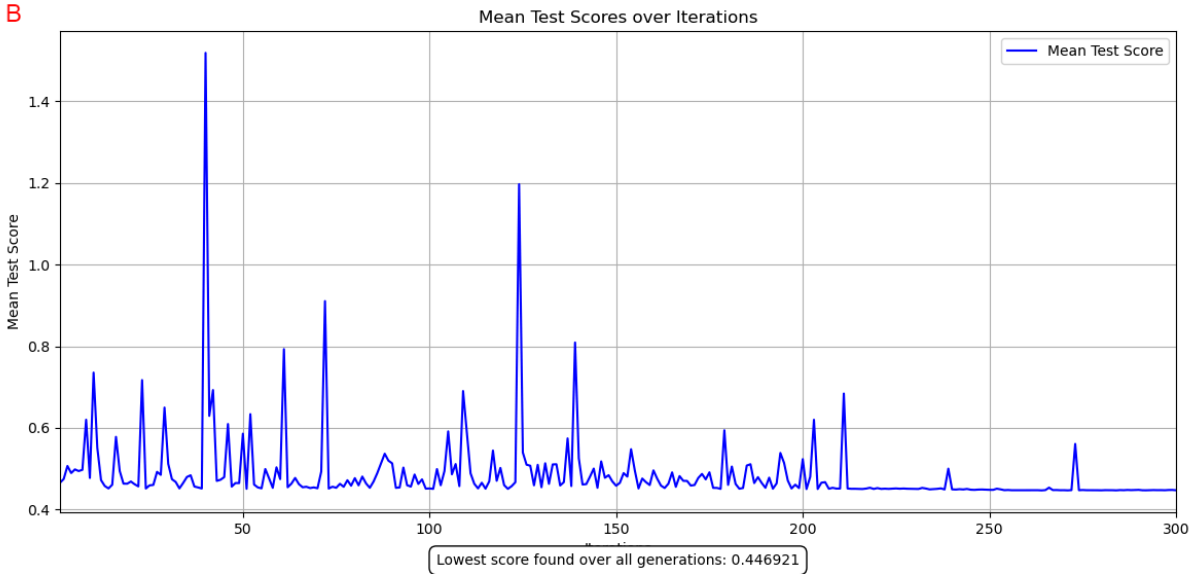
9.b.v - Medium number of iterations; large search space

Note: The first image is taken from the Melbourne dataset and the second is taken from the California dataset

A



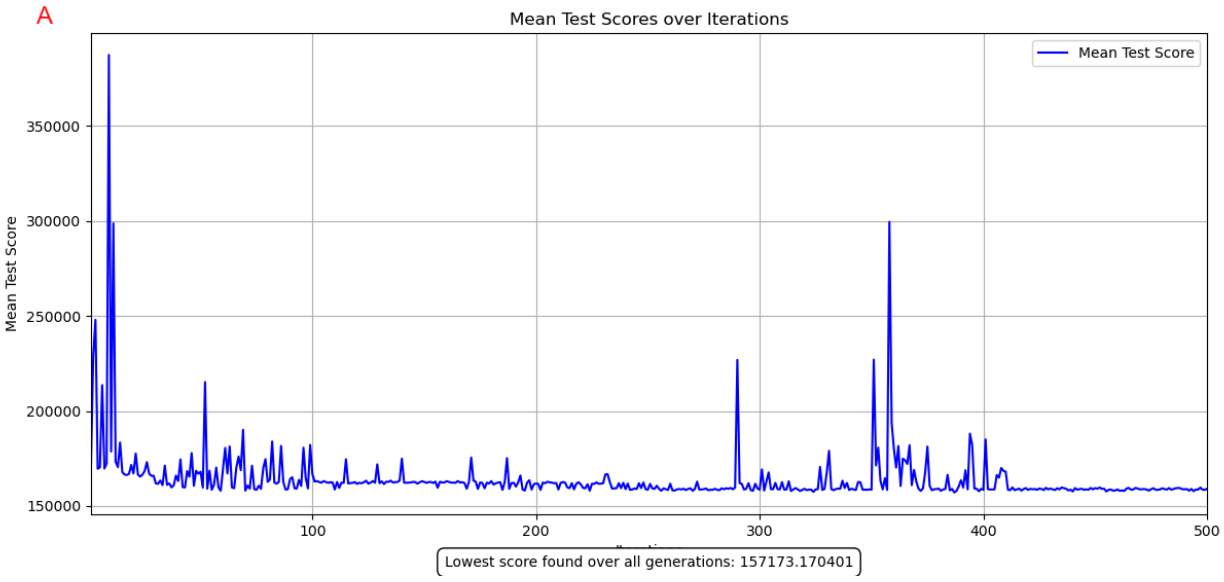
B

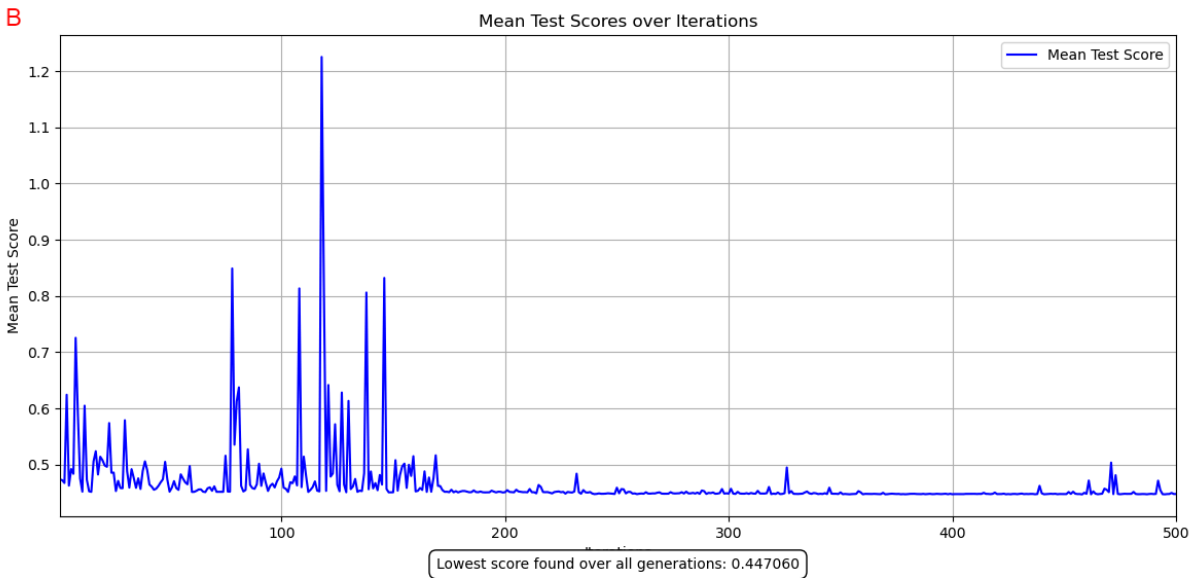


n_iter	300
Relative Lowest Fitness (Melbourne)	0.91218
Relative Lowest Fitness (California)	0.97953
Time Taken (Melbourne) (seconds)	34036.8419
Time Taken (California) (seconds)	41715.9442

9.b.vi - High number of iterations; large search space

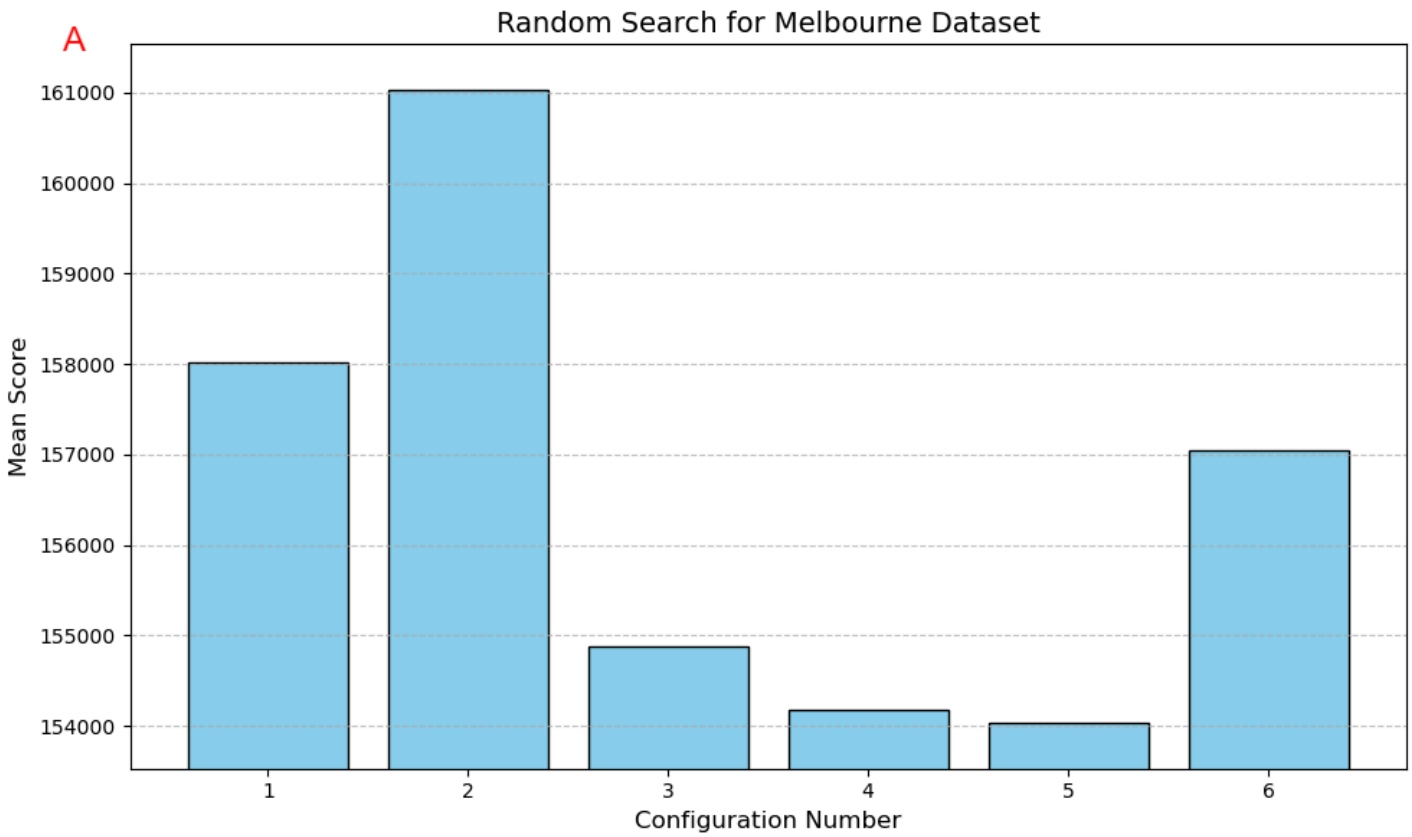
Note: The first image is taken from the Melbourne dataset and the second is taken from the California dataset

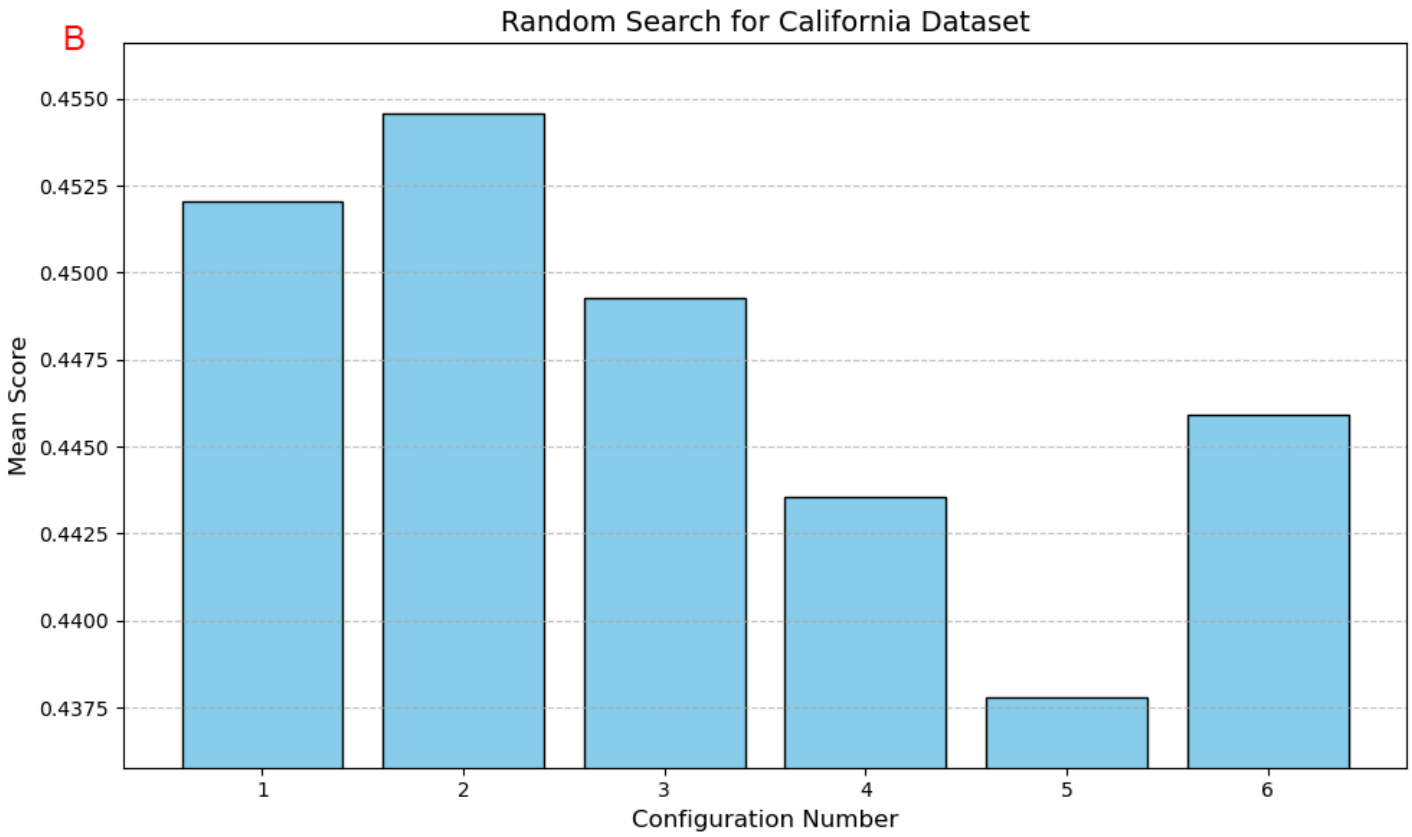




n_iter	500
Relative Lowest Fitness (Melbourne)	0.91606
Relative Lowest Fitness (California)	0.97922
Time Taken (Melbourne) (seconds)	58008.8071
Time Taken (California) (seconds)	60774.2157

9.c - Primary data collected from different configurations of RS





For small dataset

Configuration number	n_iter	Relative Lowest Fitness (Melbourne)	Relative Lowest Fitness (California)	Time Taken (Melbourne) (seconds)	Time Taken (California) (seconds)
1	1500	0.91117	0.96841	9744.7303	14238.2503
2	3000	0.89408	0.96300	19885.9717	27838.0846
3	4500	0.929585989654 3348	0.97439	38597.9316	55975.0753

For large dataset

Configuration number	n_iter	Relative Lowest Fitness (Melbourne)	Relative Lowest Fitness (California)	Time Taken (Melbourne) (seconds)	Time Taken (California) (seconds)
4	1500	0.93381	0.98700	30686.0226	28838.1219
5	3000	0.93476	1.0	59106.2514	62266.1658
6	4500	0.91676	0.98170	98603.6998	124947.4187

10 - Appendix 2

10.a - GA Implementation - Services Code

```
import random


def _calculatePrecision(num):
    '''
        Calculates the precision (number of decimal digits) for a floating
point number
    '''

    string = str(num).split('.')

    if len(string) > 1:
        # Scientific Notation with two parts. Ex: ['1', '3e-08']
        if 'e' in string[1]:
            beforeE, afterE = string[1].split('e')

            return (len(beforeE) + abs(int(afterE)))

        # For floating numbers. Ex: ['1', '1231']
        else:
            try:
                return len(string[1])
            except:
                return 0

    # Scientific Notation that has only one part. Ex: ['3e-08']
    else:
        if 'e' in string[0]:
            beforeE, afterE = string[0].split('e')

            return abs(int(afterE))
        else:
            return 0

def _calculateBinaryLength(value: int):
    '''
```

```
        Calculates the length of a binary value for an integer number
    '''
    return len(f"{value:b}")


def _convertIntToBinary(value: int, positive: bool, length: int):
    '''
        If the value is positive, the first bit represents the highest
exponent order of the binary value
        If the value is negative, the first bit represents if the value is
positive or not
    '''
    if positive:
        if value < 0:
            return ValueError("A positive value variable cannot be
negative.")
        binary = f"{value:b}"
        binary = '0' * (length - len(binary)) + binary
    else:
        if value >= 0:
            binary = f"0{value:b}"
        else:
            b1 = f"{value:b}"
            binary = f"1{b1[1:]}" # Converting a negative value

        binary = binary[0] + '0' * (length - len(binary)) + binary[1:]

    return binary


def _convertBinaryToInt(value: str, positive: bool):

    if positive:
        num = int(value, 2)
    else:
        num = int(value[1:], 2)
        if value[0] == "1":
            num *= -1

    return num
```

```
def getInfo(hyperparameters: dict):
    info = {}

    for key, value in hyperparameters.items():
        val = value[0]

        # For integers, the number of bits requried is calculated + 1.
First bit represents if the number is positive or negative
        if type(val) == int:
            val2 = value[1]
            n = _calculateBinaryLength(val2)

            if val < 0 and val2 < 0:
                info[key] = ["int", n, 0, False]
            elif val < 0 and val2 >= 0:
                info[key] = ["int", n + 1, 0, False] # n + 1 to accomodate
for the extra bit that represents the sign of the value
            else: # Both val and val2 are greater than zero
                info[key] = ["int", n, 0, True]

        # For floating point values
        elif type(val) == float:
            val2 = value[1]

            p1 = _calculatePrecision(val)
            p2  = _calculatePrecision(val2)

            max = p1 if p1 > p2 else p2

            val2 = val2 * (10**max)
            n = _calculateBinaryLength(int(val2))

            if val < 0 and val2 < 0:
                info[key] = ["float", n, max, False]
            elif val < 0 and val2 >= 0:
                info[key] = ["float", n + 1, max, False] # n + 1 to
accomodate for the extra bit that represents the sign of the value
            else: # Both val and val2 are greater than zero
```

```
                info[key] = ["float", n, max, True]

        elif type(val) == str:
            n = _calculateBinaryLength((len(value) - 1))
            info[key] = ["str", n, len(value)]

        elif type(val) == bool:
            info[key] = ["bool", 1]
        else:
            return ValueError("Incorrect datatype passed in the values for
hyperparameters.")

        (f"Info: {info}")
    return info


def encode(parameters = {}, info = {}, **kwargs):
        '''
            Decodes the hyperparameter configurations into a single string
            The entire string is mapped using the index of the list that
is provided by the user
            For integers, (fixed-point coding) they are simply converted
to a signed binary string, if the first bit is 0 it's positive and
vice-versa
            For floats, the chosen method is scaled-integer coding
            For string, each value will be given a number and that number
will be decoded to a unsigned binary string
            For booleans, one bit will be assigned. 1 is true and 0 is
false.

            For strings, the options should be passed which will passed in
kwargs
        '''
        chromosome = ""
        for key, value in parameters.items():

            data = info[key]
            datatype = data[0]
            length = data[1]
```

```
            if datatype == "int" or datatype == "float":
                exponent = data[2]
                positive = data[3]
                value = int(value * (10 ** exponent))
                chromosome += _convertIntToBinary(value=value,
length=length, positive=positive)

            elif datatype == "str":
                listOfItems = kwargs[key]
                n = listOfItems.index(value)
                chromosome += _convertIntToBinary(value=n, length=length,
positive=True)

            elif datatype == "bool":
                chromosome += '1' if value else '0'

            else:
                return ValueError("Incorrect datatype passed in the values
for hyperparameters.")

        return chromosome


def decode(chromosome="", info={}, **kwargs):

    hyperparameters = {}
    start = 0

    for key, value in info.items():

        datatype = value[0]
        length = value[1]
        end = start + length
        string = chromosome[start: end]
        start = end

        if datatype == "int":
            positive = value[3]
            hyperparameters[key] = _convertBinaryToInt(value=string,
positive=positive)
```

```
        elif datatype == "float":
            exponent = value[2]
            positive = value[3]

            string = _convertBinaryToInt(value=string, positive=positive)
            hyperparameters[key] = (float(string) / (10 ** exponent))

        elif datatype == "str":
            listOfItems = kwargs[key]
            try:
                hyperparameters[key] =
listOfItems[_convertBinaryToInt(value=string, positive=True)]
            except:
                hyperparameters[key] = "Invalid_data!"

        elif datatype == "bool":
            hyperparameters[key] = True if string else False

        else:
            return ValueError("Incorrect datatype passed in the values for
hyperparameters.")

    return hyperparameters


def flip(p):
    '''
        Returns a boolean true value with a specified probability (a
bernoulli random variable)
        A biased coin is tossed that comes up head with probability 'p'
        Taken from book by David E. Goldberg pg 60 - 70
    '''
    return random.random() < p



def generateRandomFloat(lower_limit, higher_limit, precision):

    scale_factor = 10 ** precision
```

```
    lower_limit = int(lower_limit * scale_factor)
    higher_limit = int(higher_limit * scale_factor)

    randomNum = random.randrange(lower_limit, higher_limit)

    return (randomNum / scale_factor)
```

10.b - GA Implementation - Core Functionality

```
from sklearn.metrics import mean_absolute_error
import random
from services import encode, decode, getInfo, flip, generateRandomFloat
import os
import math

class GenHyperOptimizer:
    '''
        A genetic algorithm that optimizes the hyperparameters of a
machine learning model
    '''

    # Arbitrary values given, to be refined
    _CROSSOVER_RATE = 0.7
    _MUTATION_RATE = 0.03
    _ELITISM_RATE = 0.04
    _REDUCTION_RATE = 0.00

    _MAX_POP = 0
    _ELITISM_POP = 0
    _REDUCTION_POP = 0 # The number of individuals that are going to be
reduced each generation
    _MAX_GEN = 0

    _sum_fitness = 0
    _max_fitness = 0
    _min_fitness = 0

    _fitness = []
```

```
    _best_params = {}
    _best_accuracy = 0

    _gen_count = 0

    _odd_generation = []
    _even_generation = []

    _stringHyper = {}

    # Initializes the optimizer, giving the required values
    def __init__(self, model=None, search_space=None, scoring=None,
objective=None, max_pop=30, max_gen=10, elitism_rate=0.04,
reduction_rate=0.00, iteration_number=None):
        '''
            model: A machine learning model defined in scikit-learn
            search_space: A dictionary specifying the hyperparameters to
be optimized for the model. Format is given belw
                - For real values. Note: the values passed must be int or
float
                    - 'name_of_hyperparameter': [lower_bound,
higher_bound]
                - For hyperparameters with string values. Note: All values
that must be evaluated has to be passed
                    - 'name_of_hyperparameter': ['string_alternatives']
                - For hyperparameters with boolean values. Can pass both
values unless absolutely sure. Won't impact performance much.
                    - 'name_of_hyperparameter': [True, False]
            scoring: A scoring method to calculate fitness function.
Example: mean_absolute_error
            objective:
                - min: The lower the fitness, the better the model. For
example, the lowest value for mean_absolute_error indicates an accurate
model
                - max: The higher the fitness, the better the model. For
example, the highest percentage accuracy of a model indicates a better
model
        '''
```

```
        if not model:
            raise ValueError("A model must be passed.")

        if not search_space:
            raise ValueError("A list of hyperparameters to be optimized
must be passed.")

        if not scoring:
            raise ValueError("A scoring metric must be passed.")

        if not objective:
            raise ValueError("One of the objective functions, min or max,
should be passed.")
        else:
            if objective != "min" and objective != "max":
                raise ValueError("Objective must be given as min or max to
evaluate the value of fitness function.")

        if max_pop % 2 == 1:
            raise ValueError("Maximum population can only be even.")

        # All values have been correctly passed.
        self._model = model
        self._search_space = search_space
        self._scoring = scoring
        self._objective = objective
        self._info = getInfo(hyperparameters=search_space)
        self._MAX_POP = max_pop
        self._MAX_GEN = max_gen
        self._ELITISM_RATE = elitism_rate
        self._REDUCTION_RATE = reduction_rate
        self._iteration_number = iteration_number

        # Number of individuals that are to be directly shifted to the
next gen without the influence of operators
        ELITISM_POP = math.ceil(elitism_rate * max_pop)

        # If the number of individuals is odd, it reduces 1 to make it
even
        if (ELITISM_POP % 2 == 1):
```

```
            ELITISM_POP -= 1

        self._ELITISM_POP = ELITISM_POP

        REDUCTION_POP = math.ceil(reduction_rate * max_pop)

        if (REDUCTION_POP % 2 == 1):
            REDUCTION_POP -= 1

        self._REDUCTION_POP= REDUCTION_POP


    def _single_point_crossover(self, p1, p2):

        try:
            crossover_point = random.randrange(1, (len(p1) - 1))
        except ValueError:
            crossover_point = 1

        c1 = p1[:crossover_point] + p2[crossover_point:]
        c2 = p2[:crossover_point] + p1[crossover_point:]

        return c1, c2


    def _hybrid_crossover(self, p1, p2):
        '''
            Here each hyperparameter value will go through crossover
            Taken inspiration from value-encoding crossover and
multi-point crossover
        '''

        if flip(self._CROSSOVER_RATE):
            start = 0
            c1 = ""
            c2 = ""
            for value in self._info.values():

                length = value[1]
                end = start + length
```

```
                # Each block contains a separate hyperparameter value
                p1_hp = p1[start: end]
                p2_hp = p2[start: end]
                start = end

                gc1, gc2 = self._single_point_crossover(p1_hp, p2_hp)
                c1 += gc1
                c2 += gc2

            return c1, c2
        else:
            return p1, p2


    def _mutation(self, p1):
        '''
            Thinking of bit-flip mutation but to look at other methods
        '''

        for bitPosition in range(len(p1)):
            if (flip(self._MUTATION_RATE)):
                flippedBit = '0' if p1[bitPosition] == '1' else '1'
                p1 = p1[:bitPosition] + flippedBit + p1[bitPosition + 1:]


        return p1


    def _rank_selection(self, generationData, alpha_rank, beta_rank,
intermediate=False):
        '''
            Rank-based selection. Ranking is based on the cost function
given
            Follows the linear ranking described by John Grefenstette in
his paper
            alpha_rank: Number of offsprings of the worst individual
            beta_rank: Number of offsprings of the best individual.
```

```
            In this selection, the rank of the least fit individual is
defined to be zero
        '''
        sum_probability = (alpha_rank + beta_rank) / 2.0 # Proved by the
author

        # Checks if the intermediate sums of the probabilities have been
added or not
        if not intermediate:
            length = len(generationData)

            interSum = 0 # Intermediate sum used in the roulette wheel
selection

            for i in range(length):
                probability = (alpha_rank + ((float(i) / (length - 1)) *
(beta_rank - alpha_rank))) / length # Formula is defined in the page 2 of
the paper
                generationData[i].append(probability) # Do I need to
include the probability of each individual being selected?
                interSum += probability
                generationData[i].append(interSum)

        p1 = generateRandomFloat(0.0, sum_probability, precision=5)
        p2 = generateRandomFloat(0.0, sum_probability, precision=5)

        p1_notfound = True
        p2_notfound = True

        index = 0

        while (p1_notfound or p2_notfound) and index < self._MAX_POP:
            current = generationData[index][4]

            if p1_notfound:
                if p1 < current:
                    p1 = generationData[index]
                    p1_notfound = False

            if p2_notfound:
```

```
                if p2 < current:
                    p2 = generationData[index]
                    p2_notfound = False

            index += 1

        return p1, p2


    def _printGenerationReport(self, generation, filename):
        # To generate a report based on the statistics calculated
        self._calculateStatistics(generation=generation)

        if not filename.endswith('.txt'):
            filename += '.txt'

        # Ensure the directory exists
        folder = os.path.dirname(filename)
        if folder and not os.path.exists(folder):
            os.makedirs(folder)

        statistics = (
            f"Model: {self._model}\n"
            f"Fitness Function: {self._scoring}\n"
            f"Objective: {self._objective}\n"
            f"Generation Count: {self._gen_count}\n"
            f"Highest Fitness: {self._max_fitness}\n"
            f"Lowest Fitness: {self._min_fitness}\n"
            f"Average Fitness: {self._sum_fitness /
float(self._MAX_POP)}\n"
            f"Best Chromosome: {generation[(self._MAX_POP - 1)][0]}\n"
            f"Best Hyperparameter Configuration: {generation[self._MAX_POP
- 1][1]}\n"
            f"Generation Data: {generation}"
        )

        with open(filename, 'w') as file:
            file.write(statistics)
```

```
    def _calculateStatistics(self, generation):
        # To calculate sum_fitness, min_fitness, max_fitness
        self._sum_fitness = 0
        length = len(generation)

        if self._objective == "max":
            self._max_fitness = generation[length - 1][2]
            self._min_fitness = generation[0][2]

            self._best_accuracy = self._max_fitness
            self._best_params = generation[length][1]
        else:
            self._max_fitness = generation[0][2]
            self._min_fitness = generation[length - 1][2]

            self._best_accuracy = self._min_fitness
            self._best_params = generation[length - 1][1]

        for i in range(self._MAX_POP):
            current = generation[i][2]

            self._sum_fitness += current


    def _calculateFitness(self, hyperparameters, X_train, y_train, X_test,
y_test):
        # Can make it more efficient
        model = self._model(
            **hyperparameters
        )

        model.fit(X_train, y_train)

        return self._scoring(y_test, model.predict(X_test))


    def _randomFillPopulation(self):
        '''
            Fills the first generation of population with random
hyperparameters
```

```
            Next update: To make it in such a way that the points are
spread out evenly
        '''
        for i in range(self._MAX_POP):
            hyperparameters = {}

            for key, value in self._search_space.items():
                data = self._info[key]
                datatype = data[0]

                if datatype == "int":
                    lowerLimit = value[0]
                    higherLimit = value[1]

                    hyperparameters[key] = random.randrange(lowerLimit,
higherLimit + 1)

                elif datatype == "float":
                    lowerLimit = value[0]
                    higherLimit = value[1]

                    hyperparameters[key] = generateRandomFloat(lowerLimit,
higherLimit, data[2])

                elif datatype == "str":
                    self._stringHyper[key] = value

                    hyperparameters[key] = value[random.randrange(0,
len(value))]

                elif datatype == "bool":
                    if (len(value)) == 1:
                        hyperparameters[key] = value[0]
                    else:
                        hyperparameters[key] = flip(p=0.5)

                else:
                    raise ValueError("Incorrect datatype passed in the
values for hyperparameters.")
```

```
            chromosome = encode(parameters=hyperparameters,
info=self._info, **self._stringHyper)
            fitnessScore =
self._calculateFitness(hyperparameters=hyperparameters,
X_train=self._X_train, y_train=self._y_train, X_test=self._X_test,
y_test=self._y_test)
            self._odd_generation.append([chromosome, hyperparameters,
fitnessScore])

        self._best_accuracy = self._odd_generation[0][2]


    def _decodeHyperparameters(self, c, p1, p2):
        '''
            Decodes chromosome into hypeparameters
            Checks for validity by making sure that the values lie within
the range provided by the user.
        '''

        c_decoded = decode(chromosome=c, info=self._info,
**self._stringHyper)

        for key in self._stringHyper.keys():
            if c_decoded[key] == "Invalid_data!":  # This is set in the
function, decode() of services
                if flip(0.5):
                    c_decoded[key] = p1[1][key]
                else:
                    c_decoded[key] = p2[1][key]

        for key, value in self._info.items():
            datatype = value[0]
            if datatype == "str":
                if c_decoded[key] not in self._search_space[key]:
                    if flip(0.5):
                        c_decoded[key] = p1[1][key]
                    else:
                        c_decoded[key] = p2[1][key]
            elif datatype == "bool":
```

```
                if c_decoded[key] not in value:
                    c_decoded[key] = value[0]
            else:
                if c_decoded[key] < self._search_space[key][0] or
c_decoded[key] > self._search_space[key][1]:
                    if flip(0.5):
                        c_decoded[key] = p1[1][key]
                    else:
                        c_decoded[key] = p2[1][key]

        c = encode(c_decoded, info=self._info, **self._stringHyper)
        return c, c_decoded


    def optimize(self, X_train=None, y_train=None, X_test=None,
y_test=None, alpha_rank=0.3, beta_rank=1.7):
        '''
            Runs the GA in the appropriate order
        '''
        try:
            if not X_train or y_train or X_test or y_test:
                raise ValueError("Proper training and testing datasets
have to be provided.")
        except ValueError:
            if X_train.empty or X_test.empty or y_train.empty or
y_test.empty:
                raise ValueError("Datasets provided cannot be empty.")

        print("Values are good to go!")

        self._X_train = X_train
        self._y_train = y_train
        self._X_test = X_test
        self._y_test = y_test


        print("Starting to fill the initial generation of population.")
        self._randomFillPopulation()
        self._gen_count += 1
        print("Filled initial population")
```

```
        try:
            for i in range(self._MAX_GEN):

                # Sorting the generation data
                if (self._gen_count % 2 == 0):
                    currentGen = self._even_generation.copy()
                    self._even_generation.clear()
                    nextGen = self._odd_generation
                else:
                    currentGen = self._odd_generation.copy()
                    self._odd_generation.clear()
                    nextGen = self._even_generation

                if self._objective == "max":
                    currentGen = sorted(currentGen, key=lambda x: x[2],
reverse=False)
                else:
                    currentGen = sorted(currentGen, key=lambda x: x[2],
reverse=True)

                # Transferring the best individuals into the next
generation
                # In this elitism method, even the elitist individuals are
open to mating
                for i in range(1, self._ELITISM_POP + 1):
                    nextGen.append((currentGen[-i][0:3])) # Keeps the
chromosome, the hyperparameter configuration and the fitness score.
Removes the ranking and intermediate sum

                if not self._REDUCTION_POP == 0:
                    self._MAX_POP -= self._REDUCTION_POP
                    currentGen = currentGen[self._REDUCTION_POP:] #
removes the worst individual from the population

                p1, p2 = self._rank_selection(generationData=currentGen,
intermediate=False, alpha_rank=alpha_rank, beta_rank=beta_rank)

                for index in range(int((self._MAX_POP - self._ELITISM_POP)
/ 2)):
```

```
                    c1, c2 = self._hybrid_crossover(p1=p1[0], p2=p2[0])
                    c1 = self._mutation(c1)
                    c2 = self._mutation(c2)

                    c1, c1_decoded = self._decodeHyperparameters(c=c1,
p1=p1, p2=p2)
                    c2, c2_decoded = self._decodeHyperparameters(c=c2,
p1=p1, p2=p2)


                    c1_fitness =
self._calculateFitness(hyperparameters=c1_decoded, X_train=X_train,
y_train=y_train, X_test=X_test, y_test=y_test)
                    c2_fitness =
self._calculateFitness(hyperparameters=c2_decoded, X_train=X_train,
y_train=y_train, X_test=X_test, y_test=y_test)

                    nextGen.append([c1,c1_decoded, c1_fitness])
                    nextGen.append([c2, c2_decoded, c2_fitness])

                    p1, p2
=self._rank_selection(generationData=currentGen, intermediate=True,
alpha_rank=alpha_rank, beta_rank=beta_rank)

                filename =
f"Iteration_{self._iteration_number}/Gen_{self._gen_count}.txt"
                self._printGenerationReport(currentGen, filename)
                print(f"Number of generations completed:
{self._gen_count}\nStats saved in: {filename}")

                # Updating the number of elitism population every
generation to account for
                ELITISM_POP = math.ceil(self._ELITISM_RATE *
self._MAX_POP)

                # If the number of individuals is odd, it reduces 1 to
make it even
                if (ELITISM_POP % 2 == 1):
                    ELITISM_POP -= 1
```

```
            self._ELITISM_POP = 2 if ELITISM_POP == 0 else ELITISM_POP

            self._gen_count += 1

    except Exception as e:
        print (f"Exception: {e}")

    finally:
        print(f"Best hyperparameters found: {self._best_params}")
        print(f"Accuracy: {self._best_accuracy}")

    return self._best_params
```